

Identificação

Título do Projeto: DocChain — Plataforma de Registro Documental com Prova de Integridade em Blockchain

Linha de Projeto (Direction): Web / Plataforma

Autor: *Rafael Pavesi dos Passos*

Data da Proposta: 23/06/2026

Versão: 1.0

1. Visão do Produto e Impacto (O Problema)

Este projeto resolve um problema real ou é apenas um exercício técnico?

O DocChain resolve um problema real e crescente: a necessidade de garantir autenticidade, integridade e rastreabilidade de documentos digitais em um mundo onde fraudes documentais, adulteração de arquivos e falta de mecanismos de auditoria são riscos concretos para empresas, instituições acadêmicas e órgãos públicos.

1.1 Contexto e Problema

A gestão de documentos digitais é um dos pilares de qualquer organização moderna. Contratos, certificados, laudos, diplomas, notas fiscais e relatórios circulam diariamente em formato digital, porém sem garantias reais de que não foram adulterados após sua emissão.

Quem sofre com esse problema:

- Empresas que precisam comprovar a autenticidade de contratos e documentos regulatórios
- Instituições acadêmicas que emitem diplomas e certificados suscetíveis a falsificação
- Departamentos jurídicos que dependem da integridade de provas documentais
- Profissionais autônomos que enviam propostas, relatórios e laudos a clientes

Contexto em que o problema ocorre:

- Troca de documentos entre partes que não se conhecem ou não confiam mutuamente
- Auditorias internas ou externas que precisam verificar se um documento é original
- Disputas contratuais onde a autenticidade de um documento é questionada

Como o problema é resolvido hoje:

- Assinatura digital com certificado ICP-Brasil (custosa e burocrática)
- Cartórios de registro de documentos (presencial, lento, caro)
- Envio por email com “fé de recebimento” (sem garantia técnica de integridade)
- Hash manual compartilhado por canais separados (processo frágil e não padronizado)

Limitações das soluções atuais:

- Custo elevado de certificados digitais e serviços cartoriais

- Dependência de intermediários centralizados (cartórios, certificadoras)
- Ausência de verificação pública e transparente — apenas quem possui o certificado pode validar
- Nenhuma das soluções tradicionais combina criptografia + armazenamento seguro + registro imutável em um único fluxo

1.2 Origem da Demanda e Evidências

Posicionamento do Produto:

1. O DocChain é um produto **SaaS open-source** de registro documental com blockchain, construído com a mesma arquitetura, qualidade e padrões de uma plataforma comercial real (autenticação, criptografia em repouso, API REST documentada, frontend responsivo, integração on-chain, infra containerizada)
2. A opção por permanecer na **rede de teste Sepolia** e estratégica, não acadêmica: mantém o produto **gratuito, auditável** publicamente pela comunidade e coerente com a filosofia open-source — qualquer pessoa pode clonar o repositório, rodar localmente e contribuir
3. O projeto e também entregue como **Projeto Portfólio (TCC)** da Católica SC, demonstrando competência técnica em desenvolvimento full-stack, segurança da informação e tecnologia blockchain
4. A escolha do tema foi motivada pela crescente adoção de blockchain para verificação de credenciais por governos e universidades, e pelo cenário alarmante de fraudes documentais no Brasil e no mundo

Evidência de Interesse — Adoção Institucional de Blockchain para Documentos:

A demanda por verificação de documentos via blockchain já é reconhecida por governos e instituições:

1. **Ministério da Educação (Brasil)** — lançou o programa de Diploma Digital com blockchain para toda a rede de ensino público federal, com previsão de beneficiar mais de 1,3 milhão de estudantes matriculados em cursos de graduação (fonte: Cointelegraph Brasil)
2. **UFPB (Universidade Federal da Paraíba)** — pioneira no Brasil em utilizar blockchain para registro de diplomas, criando uma rede de integridade com múltiplas camadas de validação para processos de emissão e anulação de diplomas (fonte: Livecoins)
3. **Universidade de Nicosia (Chipre)** — em 2014, tornou-se a primeira instituição no mundo a registrar certificados de conclusão de curso em blockchain (fonte: Bloomberg Linea)
4. **MIT (EUA)** — implementou o programa Digital Diplomas, permitindo que graduados recebam diplomas verificáveis em blockchain
5. **Mercado global** — o mercado de verificação de documentos cresceu de USD 4,24 bilhões em 2024 para USD 5,05 bilhões em 2025 (CAGR de 19,3%), com projeção de atingir USD 9,94 bilhões até 2029 (fonte: Research and Markets)
6. **Regulamentações** como a LGPD e o Marco Civil da Internet exigem cada vez mais rastreabilidade e integridade de dados, criando demanda por ferramentas que comprovem a não adulteração de documentos

Cenário de Fraude Documental — Por que isso importa:

A fraude documental é um problema estrutural e bilionário que reforça a urgência de mecanismos de verificação de integridade:

No Brasil:

1. A sonegação fiscal atinge cerca de **R\$ 600 bilhões por ano** (fonte: Sindifisco Nacional)
2. A Receita Federal desarticulou um esquema de fraude fiscal de **R\$ 26 bilhões** envolvendo o grupo Refit em São Paulo (fonte: Agência Brasil)
3. No Ceará, a Sefaz identificou 68 empresas de fachada que emitiram **R\$ 1,4 bilhão em notas fiscais falsas** (fonte: GC Mais)

4. No Paraná, foram identificadas **844 “empresas noteiras”** desde 2017, que emitiram notas de operações fictícias totalizando **R\$ 4,8 bilhões** (fonte: Diário dos Campos)
5. A Polícia Civil investigou um esquema que movimentou mais de **R\$ 7,6 bilhões em notas fiscais frias** utilizando empresas fictícias (fonte: Metrôpoles)

Esses números evidenciam a necessidade urgente de ferramentas que garantam a integridade e autenticidade de documentos de forma transparente e verificável — exatamente o problema que o DocChain se propõe a resolver.

Pesquisa com Usuários e Dados de Mercado:

Conversas informais com profissionais de TI, jurídico e contabilidade de pequenas e médias empresas, combinadas com dados públicos de pesquisas, revelaram um cenário preocupante:

1. **Maturidade digital baixa** — Segundo pesquisa da FGV, 66% das micro e pequenas empresas brasileiras estão nos níveis 1 e 2 de maturidade digital (18% no nível “analogico” e 48% no nível “emergente”). Isso significa que a maioria sequer possui processos digitais estruturados para gestão de documentos
2. **Integração de sistemas precária** — Apenas 41% das empresas brasileiras possuem integração entre gestão documental e outros sistemas. Empresas integradas reportam 47% mais eficiência operacional, evidenciando o custo de não adotar ferramentas adequadas
3. **Compliance desconhecido** — Dados do Sebrae indicam que apenas 29% das PMEs sabem que programas de integridade podem atenuar penas em processos de corrupção, mostrando baixo conhecimento sobre a importância da rastreabilidade documental
4. **Barreiras financeiras e de conhecimento** — 25% das PMEs apontam a falta de conhecimento sobre como conduzir a transformação digital como principal obstáculo. Empresas menores enfrentam dificuldade para adquirir e integrar novas tecnologias por limitação de recursos financeiros e humanos
5. **Dependência de certificação cara** — A única ferramenta amplamente reconhecida para verificação de integridade documental no Brasil é o certificado ICP-Brasil, que exige custos anuais de R\$ 150-500 por certificado — inviável para a maioria das PMEs

Principais dores identificadas nas conversas:

1. Ausência de mecanismo automatizado de verificação de integridade de documentos
2. Custo proibitivo de certificação digital para empresas de pequeno porte
3. Falta de histórico ou trilha de auditoria sobre documentos enviados e recebidos
4. Dependência total de terceiros centralizados (cartórios, certificadoras) para comprovar autenticidade
5. Receio de disputas contratuais sem poder provar que um documento não foi adulterado após o envio

1.3 Análise de Soluções Existentes (Benchmark)

Solução	Link	Público-Alvo	Funcionalidades Principais	Limitações
Blockcerts	blockcerts.org	Universidades e emissores de credenciais	Emissão e verificação de certificados em blockchain (Bitcoin/Ethereum)	Focado exclusivamente em credenciais acadêmicas; não suporta documentos genéricos; sem criptografia de conteúdo
OriginalMy	originalmy.com	Empresas e pessoas físicas (Brasil)	Registro de autenticidade de documentos em blockchain; assinatura digital	Serviço pago com planos mensais; código fechado; dependência total do provedor

Solução	Link	Público-Alvo	Funcionalidades Principais	Limitações
OpenTimestamps	opentimestamps.org	Desenvolvedores e entusiastas	Carimbo de tempo em blockchain Bitcoin (prova de existência)	Apenas timestamp - não armazena, não criptografa, não oferece interface amigável
Certisign	certisign.com.br	Empresas com obrigatoriedade de certificado ICP-Brasil	Assinatura digital com validade jurídica, certificados A1/A3	Custo elevado (R\$ 150-500/ano por certificado); burocrático; sem verificação pública descentralizada
DocuSign	docusign.com	Empresas globais	Assinatura eletrônica, workflow de documentos, auditoria	Centralizado; caro para PMEs; não usa blockchain; confiança depositada no provedor

Comparação:

Critério	Blockcerts	OriginalMy	OpenTimestamps	Certisign	DocuSign	DocChain
Registro em blockchain	Sim	Sim	Sim	Não	Não	Sim
Criptografia do arquivo	Não	Parcial	Não	Não	Não	Sim (AES-256-GCM)
Verificação pública	Sim	Parcial	Sim	Não	Não	Sim
Código aberto	Sim	Não	Sim	Não	Não	Sim
Documentos genéricos	Não	Sim	Sim	Sim	Sim	Sim
Interface web amigável	Limitada	Sim	Não	Sim	Sim	Sim
Custo	Gratuito	Pago	Gratuito	Pago	Pago	Gratuito (testnet)

Diferencial do Projeto:

O DocChain se diferencia por combinar em uma única plataforma open-source: 1. **Criptografia end-to-end** (AES-256-GCM) — o arquivo é armazenado criptografado, não apenas registrado 2. **Registro on-chain transparente** — qualquer pessoa pode verificar a existência de um documento na blockchain sem depender do provedor 3. **Verificação pública sem autenticação** — terceiros podem verificar a autenticidade de um documento sem precisar de conta na plataforma 4. **Stack moderna e auditável** — código aberto, arquitetura modular, e tecnologias amplamente utilizadas no mercado

Nenhuma das soluções existentes combina criptografia de conteúdo + armazenamento seguro + registro imutável + verificação pública em uma plataforma open-source com interface amigável.

1.4 Público-Alvo

Usuários primários:

- Profissionais autônomos** (advogados, contadores, consultores) que precisam registrar a autenticidade de documentos enviados a clientes, garantindo prova de integridade em caso de disputa
- Pequenas e médias empresas** que não possuem orçamento para soluções corporativas de assinatura digital (Certisign, DocuSign), mas precisam de um mecanismo confiável de verificação

Usuários secundários:

1. **Estudantes e pesquisadores** interessados em entender na prática como blockchain pode ser aplicada a problemas reais de segurança da informação
2. **Desenvolvedores** que buscam uma referência open-source de integração NestJS + Solidity + Next.js

Perfil do usuário:

1. Conhecimento técnico básico a intermediário (sabe usar navegador web, fazer upload de arquivos)
2. Não necessita conhecer blockchain — a complexidade técnica e abstraida pela interface
3. Acesso via navegador

Contexto de uso:

1. Escritório ou home office, durante o fluxo de trabalho com documentos
2. Upload pontual de documentos importantes (contratos, laudos, certificados)
3. Verificação ocasional quando a autenticidade de um documento é questionada

1.5 Objetivos do Projeto

Objetivo Geral:

Desenvolver uma plataforma web funcional que permita o registro, armazenamento seguro e verificação de autenticidade de documentos digitais, utilizando criptografia e blockchain como mecanismos de prova de integridade — demonstrando competência técnica em desenvolvimento full-stack e tecnologias emergentes.

Objetivos Específicos:

1. **Implementar um Smart Contract** em Solidity para registro imutável de hashes de documentos na rede Sepolia (testnet Ethereum), garantindo rastreabilidade e auditabilidade on-chain. A permanência na Sepolia é uma **opção estratégica do produto** — mantém o uso gratuito, alinhado com a natureza open-source da plataforma. Migração para mainnet (Ethereum, Polygon ou Arbitrum) fica disponível como evolução opcional caso usos específicos exijam permanência comercial dos registros, mas não é o objetivo desta versão
2. **Construir uma API REST** com NestJS que orquestre o fluxo completo: upload, hash SHA-256, criptografia AES-256-GCM, armazenamento seguro e registro em blockchain
3. **Desenvolver uma interface web** com Next.js 14 que abstraia a complexidade técnica, oferecendo upload intuitivo, dashboard documental e verificação de integridade com feedback visual
4. **Implementar verificação pública** que permita terceiros (sem autenticação) validarem a autenticidade de um documento comparando seu hash com o registro on-chain
5. **Documentar a arquitetura e decisões técnicas** de forma clara e completa, atendendo aos requisitos acadêmicos e servindo como peça de portfolio profissional

1.6 Métricas de Sucesso (KPIs)

Métrica	Meta	Como Medir
Fluxo completo funcional	Upload → hash → criptografia → blockchain → dashboard em um único fluxo	Teste end-to-end via interface web
Tempo de registro on-chain	Confirmação em menos de 30 segundos na Sepolia (testnet escolhida estrategicamente para manter o produto gratuito e open-source)	Medição do tempo entre envio da transação e confirmação do bloco

Métrica	Meta	Como Medir
Verificação de integridade	100% de acuracia - arquivo idêntico retorna "autentico", arquivo alterado retorna "falha"	Testes com arquivos originais e modificados (alteração de 1 byte)
Cobertura de testes do Smart Contract	100% das funções do contrato cobertas por testes automatizados	Relatório do Hardhat test coverage
API documentada	Todos os endpoints documentados via Swagger (OpenAPI)	Acesso a /api/docs com documentação completa
Criptografia correta	Arquivo descriptografado idêntico ao original (zero perda de dados)	Comparação byte-a-byte entre original e descriptografado
Uptime do protótipo	Ambiente Docker funcional com um único comando (docker compose up)	Teste de inicialização dos serviços sem erros

2. Engenharia de Requisitos

Esta seção define o que o sistema fara, evitando descrições vagas e ambiguidades. Os requisitos foram derivados da análise do problema (Capítulo 1) e do público-alvo identificado.

2.1 Personas

Persona 1 — Mariana, a Advogada Autônoma

- Idade:** 34 anos
- Profissao:** Advogada autônoma especializada em direito empresarial
- Contexto:** Atende cerca de 20 clientes pessoa fisica e jurídica; envia diariamente contratos, pareceres e procurações por email
- Objetivos:**
 - Garantir que contratos enviados não sejam alterados pelo cliente antes da assinatura
 - Ter prova jurídica de integridade caso o cliente conteste o conteúdo de um documento
 - Reduzir custos com certificação digital ICP-Brasil
- Principais dificuldades:**
 - Custo elevado de certificados digitais (R\$ 300-500/ano)
 - Falta de mecanismo simples para terceiros validarem a autenticidade
 - Receio de disputas judiciais sem prova material da versão original
- Cenário de uso DocChain:** Antes de enviar o contrato ao cliente, Mariana faz upload na plataforma. O hash fica registrado na blockchain. Caso surja disputa, ela comprova publicamente a integridade do documento original.

Persona 2 — Ricardo, o Contador de PME

- Idade:** 45 anos
- Profissao:** Contador, dono de escritório com 8 colaboradores que atende ~150 PMEs

3. **Contexto:** Lida diariamente com notas fiscais, balanços, declarações fiscais e documentos que precisam de rastreabilidade
4. **Objetivos:**
 1. Registrar de forma imutável a versão final de documentos contábeis enviados a clientes
 2. Auditar facilmente quais documentos foram enviados em determinada data
 3. Oferecer um diferencial aos clientes (verificação pública de autenticidade)
5. **Principais dificuldades:**
 1. Volume alto de documentos torna inviável certificar tudo via ICP-Brasil
 2. Clientes ocasionalmente alegam ter recebido versão diferente do documento
 3. Falta de trilha de auditoria automatizada
6. **Cenário de uso DocChain:** Ricardo integra o DocChain ao fluxo do escritório. Cada documento enviado e registrado on-chain. Em fiscalizações ou auditorias, basta apresentar o hash e o link da transação.

Persona 3 — Bianca, a Estudante de TI

1. **Idade:** 22 anos
2. **Profissão:** Estudante de Engenharia de Software em fase final do curso
3. **Contexto:** Estuda aplicações práticas de blockchain e busca referências de projetos open-source para basear seu próprio TCC
4. **Objetivos:**
 1. Entender na prática como integrar Smart Contracts a um backend tradicional
 2. Estudar arquitetura modular com NestJS, Prisma e Next.js
 3. Usar o código como base para seu próprio projeto de portfolio
5. **Principais dificuldades:**
 1. Escassez de projetos open-source que combinem Web3 com stack moderna de mercado
 2. Tutoriais cobrem apenas partes isoladas (só contrato, só backend)
 3. Dificuldade em encontrar exemplos didáticos mas também realísticos
6. **Cenário de uso DocChain:** Bianca clona o repositório, sobe o ambiente com docker compose up, lê a documentação técnica e estuda como o sistema orquestra hash + criptografia + on-chain.

2.2 Casos de Uso Principais

O sistema possui três atores principais e os seguintes casos de uso:

Atores:

1. **Usuário Autenticado** — pessoa registrada na plataforma com acesso ao dashboard
2. **Visitante (Verificador Público)** — qualquer pessoa que queira validar a autenticidade de um documento sem ter conta
3. **Sistema Blockchain (Sepolia)** — ator externo que recebe e registra hashes de forma imutável

Casos de Uso:

ID	Caso de Uso	Ator Principal	Descrição Resumida
UC01	Criar Conta	Visitante	Cadastra-se com email, senha e nome para acessar a plataforma
UC02	Realizar Login	Usuário	Autentica-se com email e senha, recebe JWT
UC03	Fazer Upload de Documento	Usuário Autenticado	Envia arquivo que sera hashado, criptografado e registrado on-chain
UC04	Visualizar Dashboard	Usuário Autenticado	Consulta lista paginada de documentos com status e metadados
UC05	Visualizar Detalhe do Documento	Usuário Autenticado	Consulta metadados completos, hash, txHash e link Etherscan
UC06	Verificar Integridade (Privada)	Usuário Autenticado	Reenvia o arquivo para comparar hash com o registro original
UC07	Baixar Documento Descriptografado	Usuário Autenticado	Recupera o arquivo original a partir do storage criptografado
UC08	Verificar Documento Publicamente	Visitante	Sem autenticação, consulta a blockchain via hash ou arquivo
UC09	Registrar Hash on-chain	Sistema -> Blockchain	Envia transação para o Smart Contract na Sepolia
UC10	Confirmar Registro on-chain	Blockchain -> Sistema	Sistema aguarda confirmação de bloco e atualiza status



Diagrama de Casos de Uso (UML) — atores, boundary DocChain e 10 casos de uso

2.3 Requisitos Funcionais (RF)

Módulo de Autenticação:

1. **RF01** — O sistema deve permitir que o Visitante crie uma conta informando email, senha e nome.

2. **RF02** — O sistema deve permitir que o Usuário realize login com email e senha, recebendo um token JWT.
3. **RF03** — O sistema deve invalidar tokens JWT após 15 minutos, exigindo novo login para sessões prolongadas.
4. **RF04** — O sistema deve permitir que o Usuário realize logout, descartando o token no lado do cliente.

Módulo de Upload e Registro:

1. **RF05** — O sistema deve permitir que o Usuário Autenticado faça upload de arquivos de qualquer tipo (PDF, DOCX, imagens, etc.) com tamanho máximo de 50 MB.
2. **RF06** — O sistema deve calcular o hash SHA-256 do arquivo **original** (antes da criptografia) automaticamente após o upload.
3. **RF07** — O sistema deve criptografar o conteúdo do arquivo utilizando AES-256-GCM antes de armazená-lo em disco.
4. **RF08** — O sistema deve registrar o hash SHA-256 do documento no Smart Contract da rede Sepolia, junto com a referência de storage.
5. **RF09** — O sistema deve persistir os metadados do documento (id, nome, tipo, tamanho, hash, txHash, status, datas, IV, authTag) no banco PostgreSQL.
6. **RF10** — O sistema deve impedir o registro duplicado de hashes já existentes no contrato, retornando erro específico.

Módulo de Dashboard e Consulta:

1. **RF11** — O sistema deve apresentar ao Usuário Autenticado uma lista paginada (10 itens por pagina) dos seus documentos.
2. **RF12** — O sistema deve exibir cards de estatísticas com total de documentos, confirmados e pendentes.
3. **RF13** — O sistema deve permitir filtrar a lista por status (PENDING, PROCESSING, CONFIRMED, FAILED).
4. **RF14** — O sistema deve permitir o Usuário Autenticado consultar o detalhe completo de cada documento.
5. **RF15** — O sistema deve exibir link clicável para a transação no Etherscan Sepolia.

Módulo de Verificação:

1. **RF16** — O sistema deve permitir que o Usuário Autenticado verifique a integridade de um documento, reenviando-o para comparação de hash.
2. **RF17** — O sistema deve permitir que o Visitante (sem autenticação) verifique publicamente a autenticidade de um documento em /verify, informando o hash diretamente ou enviando o arquivo.
3. **RF18** — O sistema deve retornar resultado claro (Autentico / Falha) com dados blockchain (data, bloco, endereço que registrou).

Módulo de Download:

1. **RF19** — O sistema deve permitir que o Usuário Autenticado faça download do arquivo descriptografado original.
2. **RF20** — O sistema deve negar acesso a documentos que não pertencem ao usuário autenticado (retornar 403 ou 404).

Utilitários:

1. **RF21** — O sistema deve expor um endpoint GET /health para verificação de saúde (banco + conexão blockchain).
2. **RF22** — O sistema deve documentar todos os endpoints públicos via Swagger (OpenAPI) em /api/docs.

2.4 Requisitos Não Funcionais (RNF)

Desempenho:

1. **RNF01** — O tempo de resposta da API para operações de leitura deve ser inferior a 300 ms (excluindo operações blockchain).
2. **RNF02** — A confirmação do registro on-chain deve ocorrer em menos de 30 segundos na rede Sepolia.
3. **RNF03** — O sistema deve suportar pelo menos 20 uploads simultâneos sem degradação perceptível.

Segurança:

1. **RNF04** — Senhas devem ser armazenadas com bcrypt (cost factor ≥ 10).
2. **RNF05** — Tokens JWT devem ser assinados com chave de no mínimo 32 caracteres armazenada em variável de ambiente.
3. **RNF06** — Arquivos devem ser criptografados em repouso com AES-256-GCM utilizando IV aleatório para cada operação.
4. **RNF07** — A chave de criptografia (ENCRYPTION_KEY) nunca deve ser hardcoded — sempre via variável de ambiente.
5. **RNF08** — A chave privada da carteira blockchain (PRIVATE_KEY) deve ser armazenada apenas em variável de ambiente, nunca em código ou banco.
6. **RNF09** — Acessos a documentos devem ser restritos ao usuário proprietário (isolamento por userId).

Disponibilidade:

1. **RNF10** — O ambiente de desenvolvimento deve subir completamente com um único comando (docker compose up).
2. **RNF11** — Falhas na conexão com a blockchain devem ser tratadas com mensagens claras, sem comprometer o estado dos demais documentos.
3. **RNF12** — O sistema deve realizar rollback do registro local em caso de falha definitiva na transação blockchain.

Escalabilidade:

1. **RNF13** — A camada de storage deve ser abstraída via interface (IStorageService) para permitir troca futura de Local por IPFS sem refactor.
2. **RNF14** — A arquitetura deve permitir escalar horizontalmente o backend (stateless, JWT em vez de sessão em memória).

Usabilidade:

1. **RNF15** — A interface deve ser responsiva (mobile-first), funcionando em telas a partir de 360 px.
2. **RNF16** — A complexidade técnica de blockchain deve ser abstraída — o usuário nunca precisa entender carteiras, gas ou ABIs.
3. **RNF17** — Feedback visual claro em todas as operações (spinners, toasts, badges de status).

Manutenibilidade:

1. **RNF18** — O código deve seguir padrões ESLint + Prettier com TypeScript em modo strict.
 2. **RNF19** — O Smart Contract deve possuir 100% de cobertura de testes automatizados.
 3. **RNF20** — A API deve ter documentação Swagger automatizada e atualizada a cada deploy.
-

2.5 Regras de Negócio

1. **RN01** — Cada hash SHA-256 só pode ser registrado uma única vez no Smart Contract. Tentativas de registro duplicado retornam erro DocumentAlreadyRegistered.
2. **RN02** — O hash registrado on-chain corresponde ao arquivo **original** (antes da criptografia), de modo que a verificação possa ser feita sem descriptografar.
3. **RN03** — Documentos pertencem exclusivamente ao usuário que os registrou — não há compartilhamento na versão inicial.
4. **RN04** — A verificação pública (/verify) não expoe metadados sensíveis (nome do arquivo, dono, etc.) — apenas confirma se o hash existe na blockchain e seus dados públicos (data, endereço que registrou).
5. **RN05** — Apenas usuários autenticados podem fazer upload e gerenciar documentos.
6. **RN06** — O download do arquivo descriptografado é permitido apenas ao usuário proprietário.
7. **RN07** — Documentos com status FAILED podem ser reenviados (tentativa de re-registro), desde que o hash não tenha sido confirmado on-chain em transação anterior.
8. **RN08** — O tamanho máximo permitido por arquivo é de 50 MB.
9. **RN09** — Apenas hashes SHA-256 válidos (64 caracteres hexadecimais) são aceitos no endpoint de verificação pública.

2.6 Fora do Escopo

Para manter o escopo factível e coerente com a proposta de **produto SaaS open-source rodando em testnet**, os seguintes itens **não serão** implementados nesta primeira versão:

1. Assinatura digital com certificado ICP-Brasil — DocChain trabalha apenas com hash + blockchain, sem integração com cartórios ou certificadoras
2. Multiusuario / Compartilhamento de documentos — cada usuário gerencia apenas seus próprios documentos
3. Multiempresa / Multi-tenant — sem isolamento de organizações
4. Integração com sistemas externos (SAP, TOTVS, ERPs em geral)
5. Storage remoto IPFS — fica como evolução futura (a interface já prepara o caminho)
6. Migração para Mainnet — **por opção estratégica**, o DocChain opera apenas em testnet Sepolia, mantendo uso gratuito e a natureza open-source; mainnet fica como evolução opcional
7. Aplicativo mobile nativo — somente interface web responsiva
8. Modelo de monetização (cobrança / billing / assinatura) — **por opção estratégica**, o DocChain é open-source e gratuito; billing fica como caminho futuro caso a comunidade evolua o produto comercialmente
9. Notificações por email ou push — sem serviço de envio configurado
10. Recuperação de senha por email — fluxo simplificado, sem SMTP
11. 2FA / MFA — autenticação com email + senha + JWT apenas
12. Auditoria avançada / Logs estruturados — apenas logging básico
13. Painel administrativo — sem área admin para gestão de usuários
14. Relatórios e exportações — sem export PDF, CSV ou Excel

Esses itens compõem o **backlog de evolução futura** — caso a comunidade open-source contribua ou surjam casos de uso específicos, podem ser explorados em iterações posteriores sem comprometer a arquitetura atual (que já foi desenhada com extensibilidade em mente: IStorageService abstrato, backend stateless, módulos independentes).

3. Fluxos e Comportamento do Sistema

Esta seção demonstra como o sistema funciona em seus fluxos principais e tratamentos de erro.

3.1 Fluxo Principal — Registro de Documento

O fluxo principal do DocChain é o registro de um documento na blockchain, executado em 12 etapas:

1. **Login** — Usuário acessa /login, informa email e senha, recebe JWT armazenado em httpOnly cookie
2. **Acesso ao upload** — Usuário navega para /upload
3. **Seleção do arquivo** — Usuário arrasta arquivo na dropzone ou clica para selecionar (max. 50 MB)
4. **Confirmação** — Usuário clica em “Registrar na Blockchain”
5. **Envio multipart** — Frontend envia POST /documents com arquivo + Authorization Bearer JWT
6. **Validação backend** — JwtAuthGuard valida token; Multer carrega arquivo em buffer
7. **Criação do registro** — DB persiste registro inicial com status PENDING
8. **Calculo do hash** — CryptoService.hashFile(buffer) gera SHA-256 do arquivo original
9. **Criptografia** — CryptoService.encrypt(buffer) gera ciphertext + IV + authTag (AES-256-GCM)
10. **Armazenamento** — StorageService.save(ciphertext, hash) grava em /uploads/{hash}.enc
11. **Registro on-chain** — BlockchainService.registerDocument(hash, storageRef) chama o contrato e aguarda 1 confirmação
12. **Atualização final** — Status = CONFIRMED; persiste txHash, blockNumber, encryptionIv, encryptionAuthTag

Após a confirmação, o frontend atualiza a UI mostrando o hash truncado, link Etherscan e badge “Confirmado”.

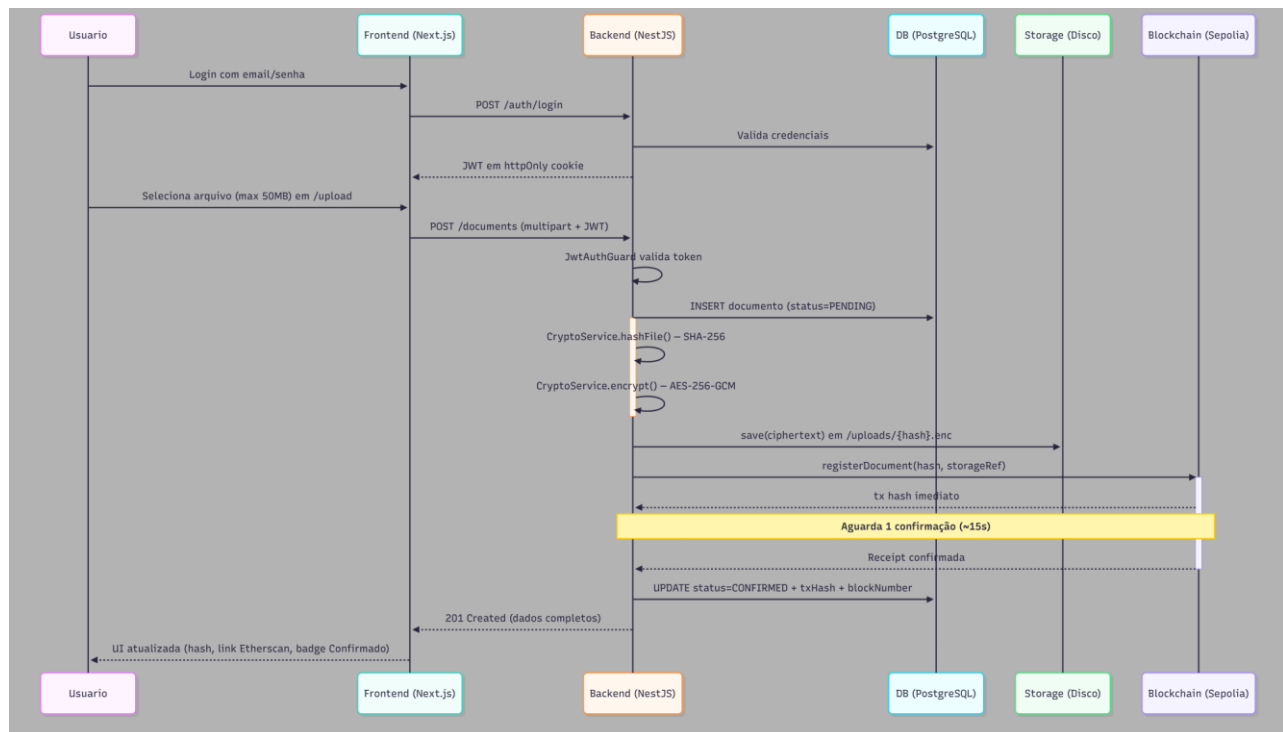
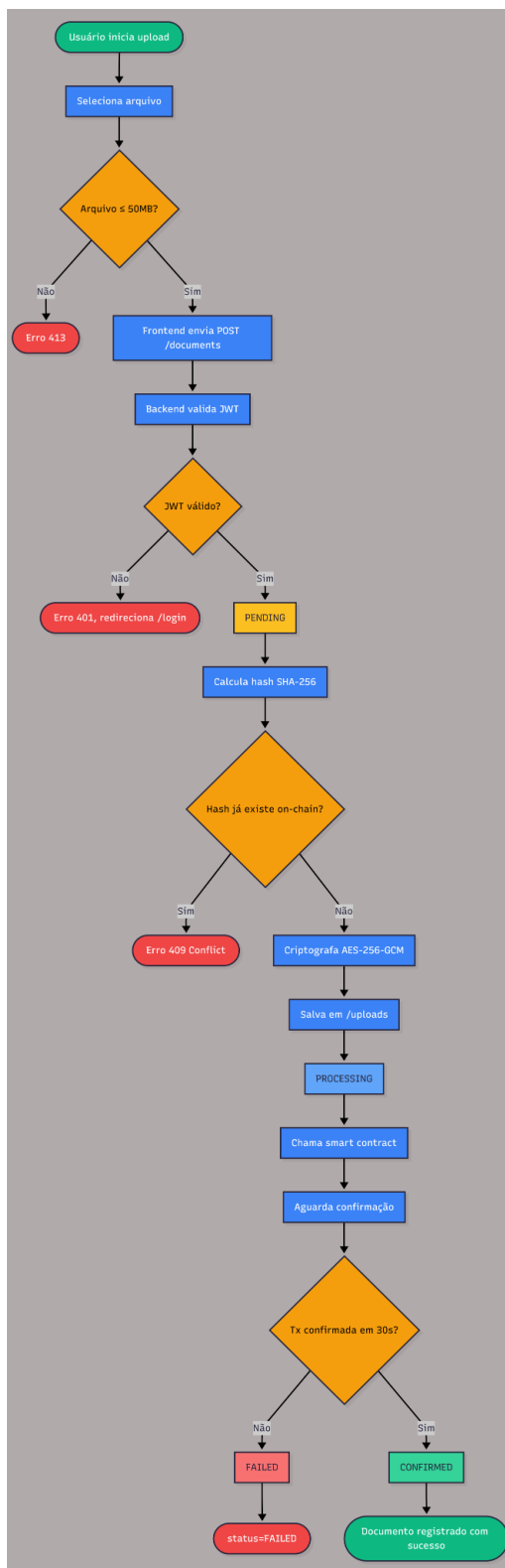


Diagrama de Sequência — fluxo principal de upload e registro on-chain

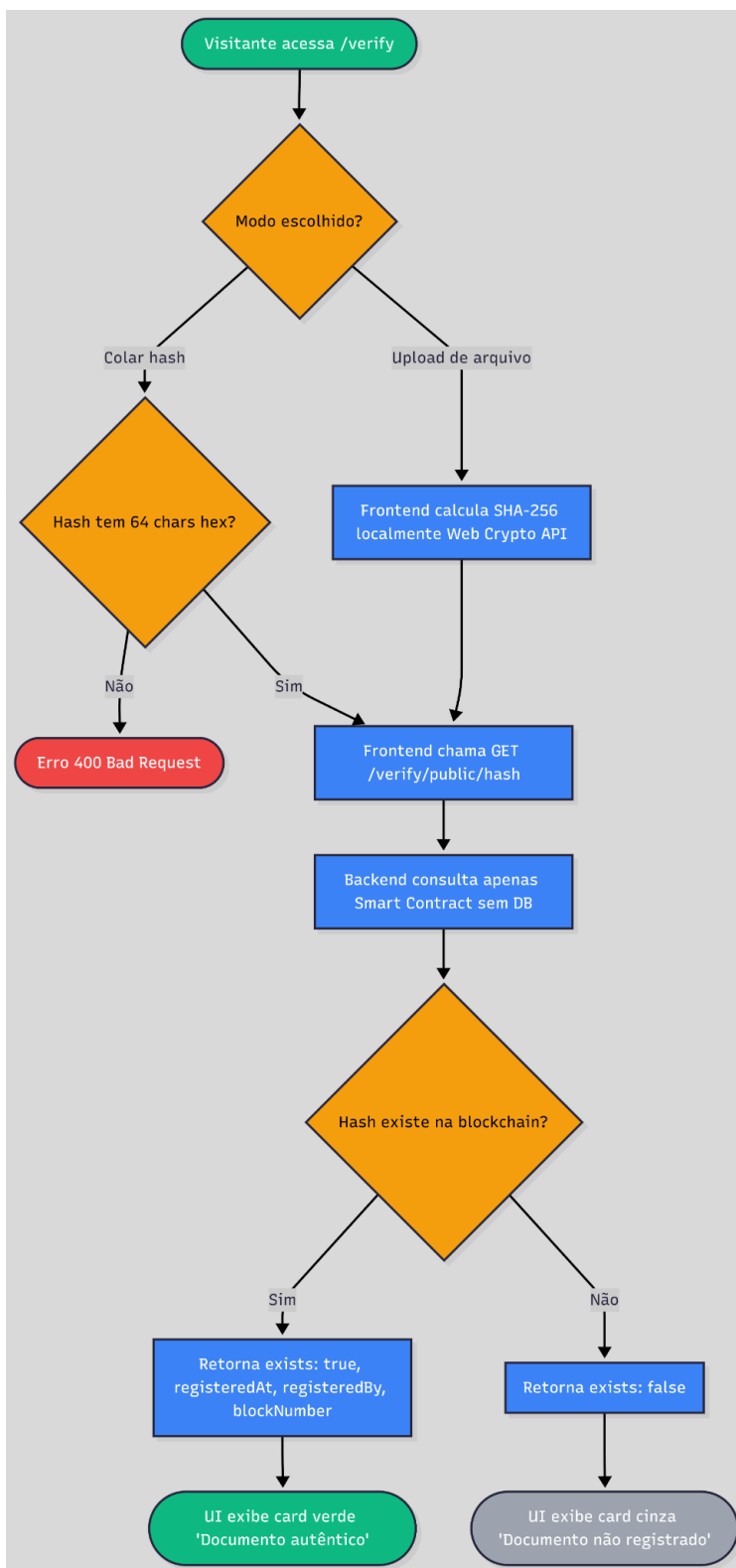


Fluxograma — fluxo principal de upload e registro

3.2 Fluxo Alternativo — Verificação Pública

A verificação pública permite a um terceiro validar a autenticidade de um documento sem possuir conta na plataforma:

1. Visitante acessa /verify (rota pública, sem autenticação)
2. Escolhe um dos modos:
 1. **Modo A:** Faz upload de arquivo → frontend calcula hash SHA-256 localmente (no navegador) ou envia ao backend
 2. **Modo B:** Cola um hash SHA-256 (64 caracteres hex)
3. Clica em “Verificar”
4. Frontend chama GET /verify/public/{hash}
5. Backend consulta apenas o Smart Contract (verifyDocument(hash)) — sem acessar o banco
6. Retorna:
 1. Se registrado: { exists: true, registeredAt, registeredBy, blockNumber }
 2. Se não registrado: { exists: false }
7. Frontend exibe resultado público (sem expor metadados privados como nome do dono)



Fluxograma — fluxo de verificação pública

3.3 Fluxos Alternativos — Exceções e Erros

Código	Situação	Causa	Tratamento
E01	Falha na transação blockchain durante upload	Timeout, sem saldo de ETH, contrato pausado	Status do documento marcado como FAILED; arquivo criptografado mantido em disco; toast de erro com motivo; permite retry
E02	Hash duplicado	Usuário tenta registrar arquivo cujo hash já existe no contrato	Contrato retorna DocumentAlreadyRegistered; backend responde 409 Conflict; frontend exibe mensagem clara
E03	Arquivo acima de 50 MB	Upload excede o limite	Backend responde 413 Payload Too Large antes de processar; frontend valida tamanho antes do envio
E04	Token JWT expirado	Sessao maior que 15 min	Backend responde 401; axios interceptor redireciona automaticamente para /login
E05	Tentativa de acessar documento de outro usuário	userId não confere	Backend responde 404 (intencionalmente vago, para não expor existência); frontend exibe "Documento não encontrado"
E06	Falha na descrição	Chave alterada ou arquivo corrompido	Backend responde 500 com mensagem genérica; logs internos detalham o erro real
E07	Hash invalido na verificação pública	Formato incorreto (não tem 64 hex chars)	Backend responde 400 Bad Request com mensagem explicativa

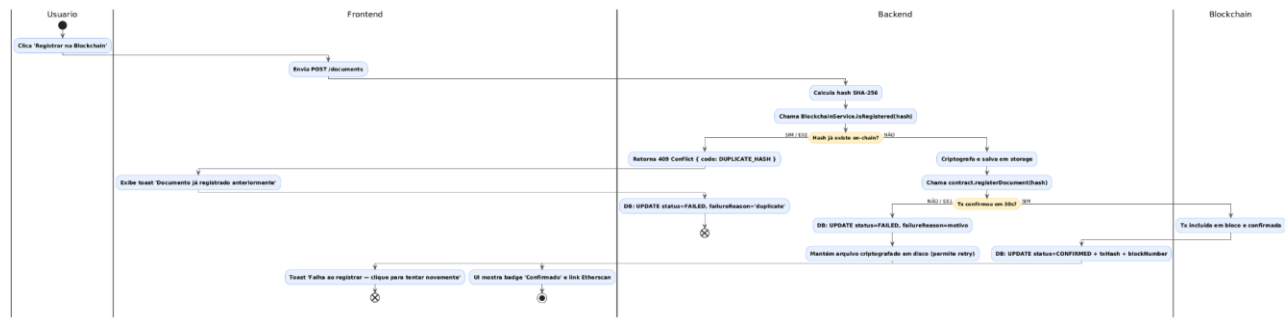


Diagrama de Atividade — tratamento de erros E01 (falha blockchain) e E02 (hash duplicado)

4. Mockups e Experiência do Usuário (UX)

Esta seção apresenta a visualização do produto antes da implementação, validando o fluxo de navegação, a organização da interface e a clareza da experiência.

4.1 Fluxo de Navegação

O DocChain possui as seguintes rotas:

Rotas Públicas (sem autenticação):

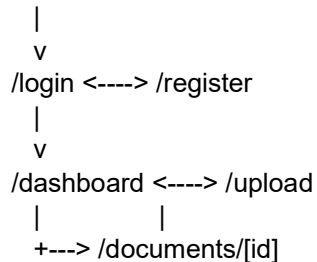
1. /login — Tela de login
2. /register — Tela de cadastro
3. /verify — Verificação pública

Rotas Protegidas (dashboard):

1. /dashboard — Lista de documentos do usuário
2. /upload — Upload de novo documento
3. /documents/[id] — Detalhe + verificação + download de um documento

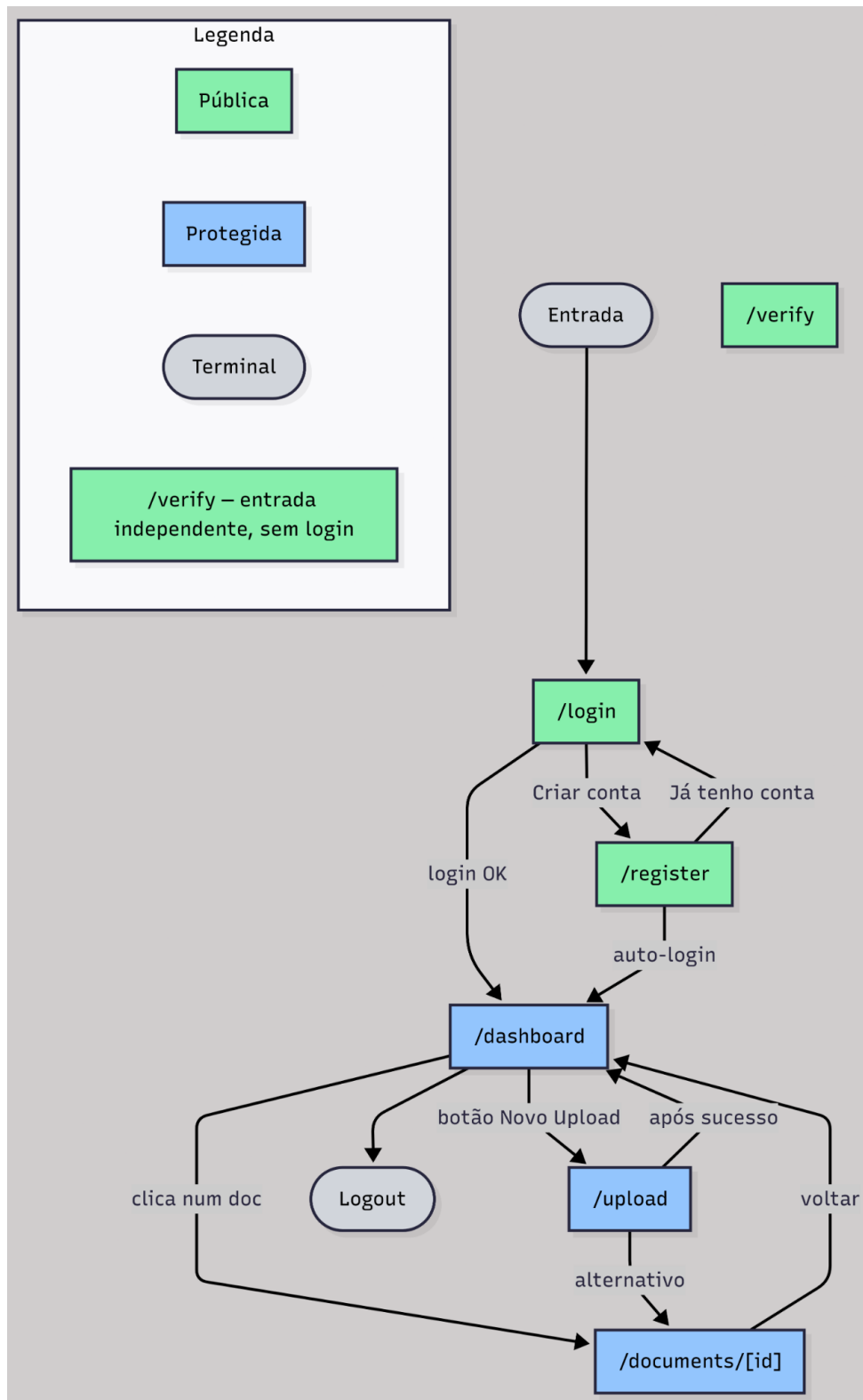
Estrutura de navegação:

[Entrada]



[Acesso público independente]

/verify



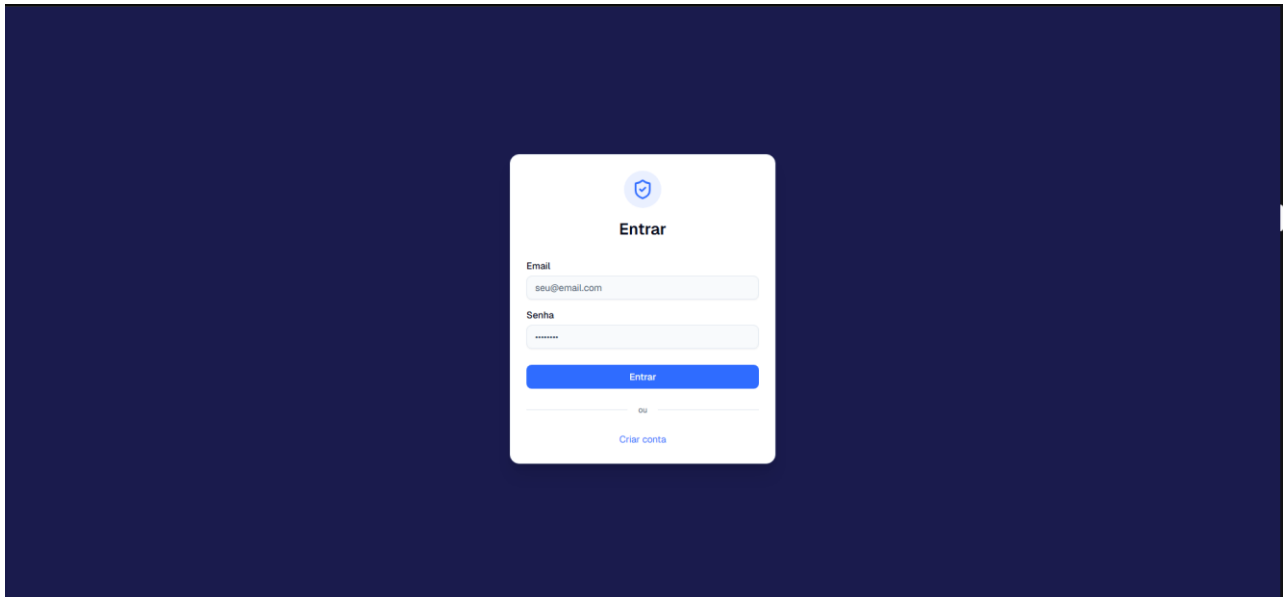
Fluxo de Navegação entre telas

4.2 Wireframes e Mockups das Telas

A seguir, descrição das principais telas. Cada uma deve ser produzida no **Figma** (recomendado) com layout responsivo (desktop 1440 px e mobile 375 px).

Tela 1: /login

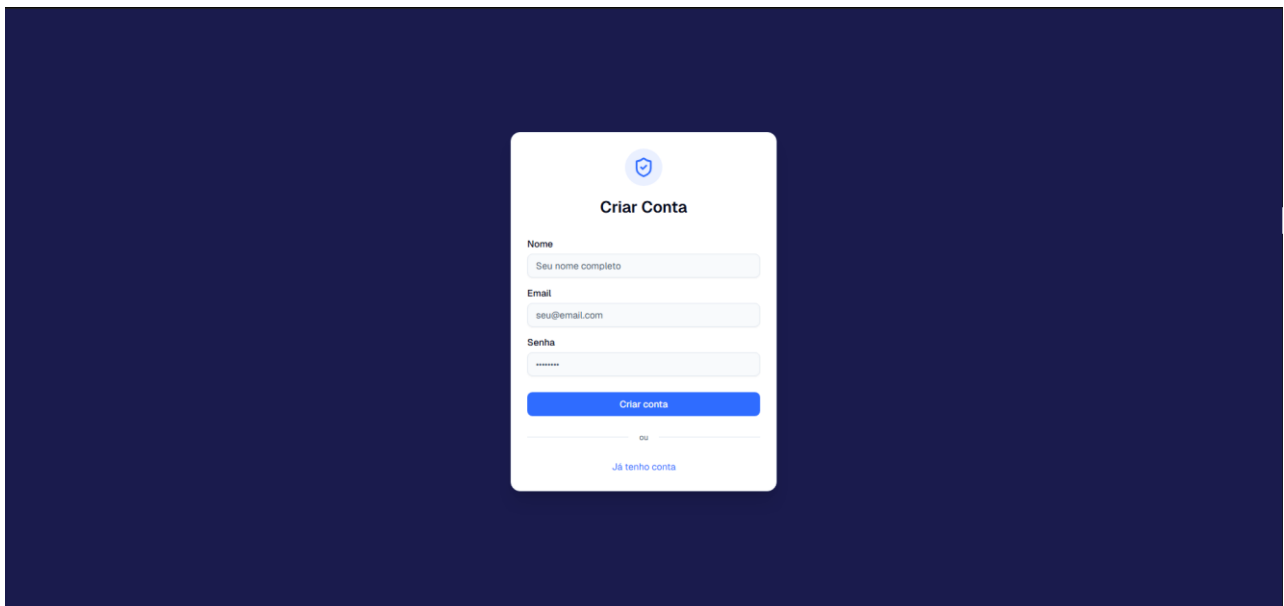
1. **Funcionalidade:** Autenticar usuário existente
2. **Componentes:** Logo, título “Entrar”, campo email, campo senha, botão “Entrar”, link “Criar conta”
3. **Ações:** Submeter formulario → POST /auth/login → recebe JWT → redireciona para /dashboard
4. **Estados:** Loading no botão durante request; toast de erro em credenciais invalidas



Mockup da tela /login

Tela 2: /register

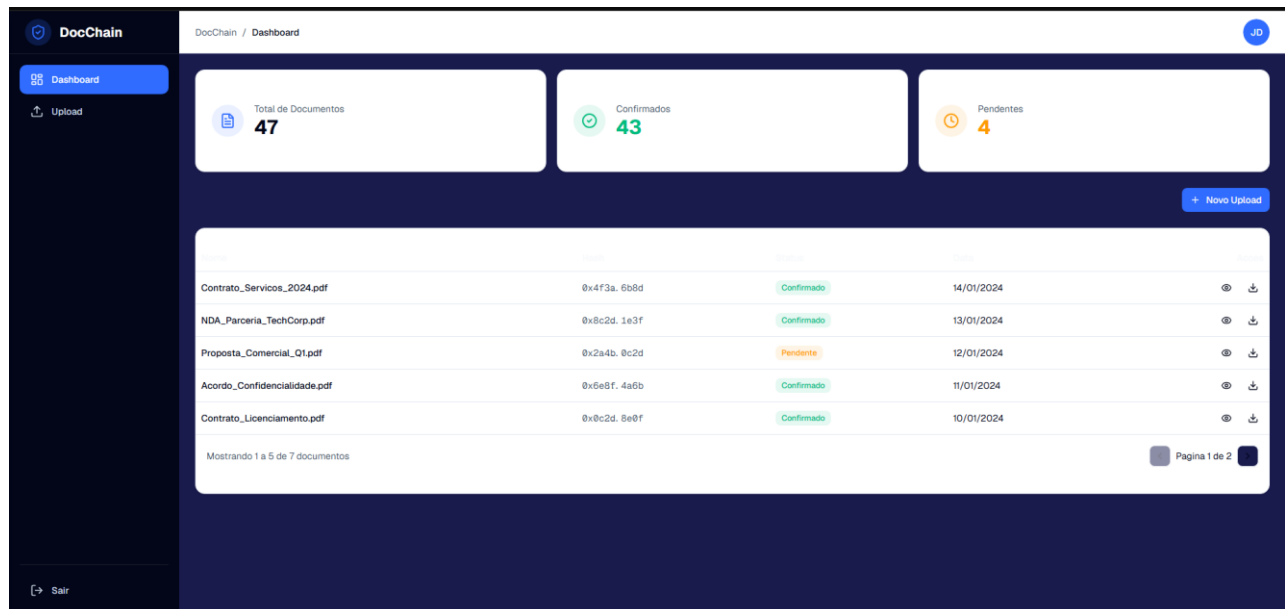
1. **Funcionalidade:** Criar nova conta
2. **Componentes:** Campos email, senha, nome, botão “Criar conta”, link “Já tenho conta”
3. **Ações:** POST /auth/register → auto-login → redireciona para /dashboard



Mockup da tela /register

Tela 3: /dashboard

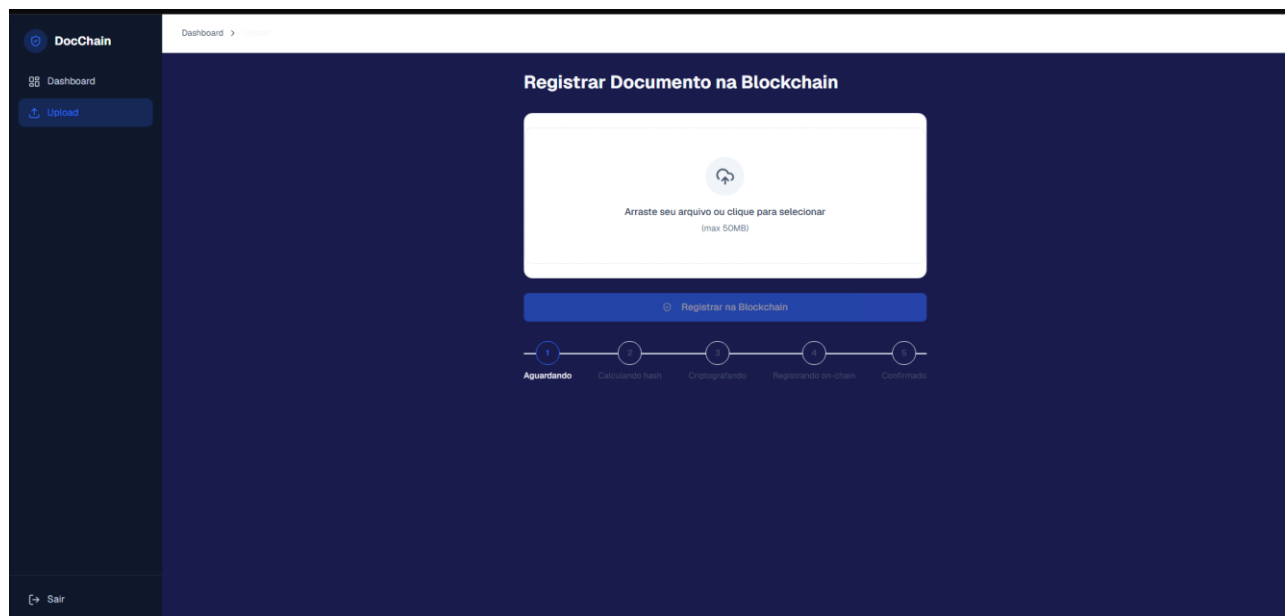
1. **Funcionalidade:** Visão geral dos documentos do usuário
2. **Componentes:**
 1. Sidebar (desktop) ou Navbar (mobile) com itens: Dashboard, Upload, Sair
 2. 3 cards de estatísticas: Total / Confirmados / Pendentes
 3. Tabela paginada com colunas: Nome, Hash truncado, Status (badge colorido), Data, Ações
 4. Botao “Novo Upload” em destaque
 5. **Empty state:** ilustração + botao “Fazer primeiro upload”
3. **Estados:** Skeleton durante load; mensagem de erro em caso de falha



Mockup da tela /dashboard

Tela 4: /upload

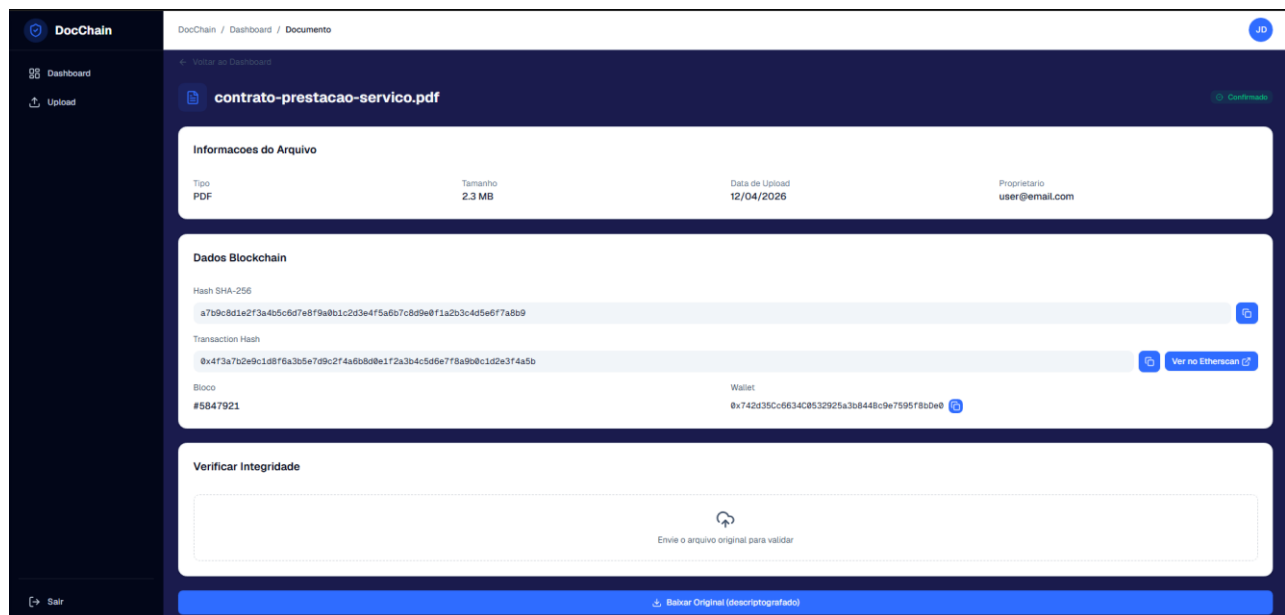
1. **Funcionalidade:** Subir e registrar um documento
2. **Componentes:**
 1. Dropzone grande (drag-and-drop ou click)
 2. Preview do arquivo selecionado (nome, tamanho, tipo)
 3. Botao “Registrar na Blockchain”
 4. **Stepper visual** mostrando os estados: Aguardando → Calculando hash → Criptografando → Registrando on-chain → Confirmado
3. **Estados:** idle → uploading → processing → confirming → success/error
4. **Tela de sucesso:** card com hash completo (botao copiar), link Etherscan, botao “Ver no Dashboard”



Mockup da tela /upload

Tela 5: /documents/[id]

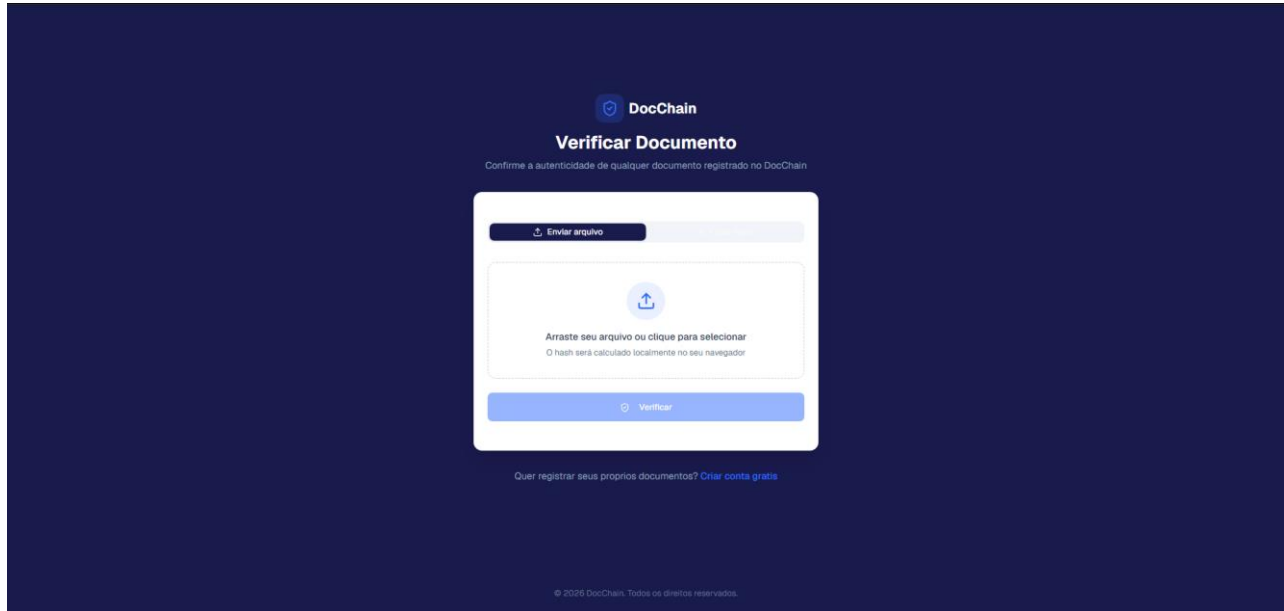
1. **Funcionalidade:** Detalhe completo do documento + verificação + download
2. **Componentes:**
 1. Header: nome do arquivo + badge de status
 2. Grid 2 colunas com metadados (tipo, tamanho, data, dono)
 3. Bloco “Blockchain”: hash completo, txHash, link Etherscan, bloco, wallet
 4. Botao “Copiar Hash”
 5. Seção “Verificar Integridade”: dropzone inline
 6. Resultado da verificação: card verde (autentico) ou vermelho (falha)
 7. Botao “Download Original”



Mockup da tela /documents/[id]

Tela 6: /verify (pública)

1. **Funcionalidade:** Validar autenticidade de qualquer documento sem login
2. **Componentes:**
 1. Layout limpo, sem sidebar
 2. Toggle entre “Enviar arquivo” e “Colar hash”
 3. Campo de input (arquivo ou hash)
 4. Botao “Verificar”
 5. Resultado: card verde (autentico, com dados blockchain) ou cinza (não encontrado)
 6. Link para /register convidando o visitante a se cadastrar



Mockup da tela /verify (pública)

4.3 Fluxo de Interação do Usuário

Demonstração passo a passo do fluxo principal (cadastro + registro de documento + verificação):

1. Usuário acessa o sistema em /login
2. Como não tem conta, clica em “Criar conta” → /register
3. Preenche email, senha, nome e clica em “Criar conta”
4. É auto-logado e redirecionado para /dashboard, que mostra empty state
5. Clica em “Novo Upload” → navega para /upload
6. Arrasta um PDF para a dropzone (ou clica e seleciona)
7. Confere preview com nome e tamanho do arquivo
8. Clica em “Registrar na Blockchain”
9. Acompanha visualmente o stepper: hash → criptografia → on-chain
10. Após ~20 segundos, ve a tela de sucesso com hash + link Etherscan
11. Clica em “Ver no Dashboard” e ve o documento listado com status “Confirmado”
12. Clica no documento → navega para /documents/[id]

13. Le os metadados, copia o hash, abre o link Etherscan em nova aba
14. Na seção “Verificar Integridade”, faz upload do mesmo arquivo → recebe resultado **autentico**
15. (Teste negativo) Altera 1 byte do arquivo e tenta verificar → recebe resultado **falhou**

Nota: a descrição sequencial dos 15 passos acima, combinada com os mockups das 6 telas (seção 4.2) e o diagrama de Fluxo de Navegação (seção 3), cumpre o requisito de “sequência de telas ou fluxo visual” do template de RFC da Católica SC.

4.4 Feedback Inicial de Usuários (Opcional)

Após a produção dos mockups, sugere-se conduzir uma validação rápida com **3 a 5 representantes do público-alvo** (advogados, contadores, estudantes) através de:

1. Apresentação do protótipo navegável do Figma
2. Roteiro de 5 tarefas: criar conta, fazer upload, ver detalhe, verificar, baixar
3. Anotação de pontos de fricção em cada tarefa
4. Coleta de sugestões via formulário curto (Google Forms ou Typeform gratuito)

Nota: sessão opcional segundo o template de RFC da Católica SC. Pode ser executada após a produção dos mockups (já concluída) como reforço de validação, sem impacto no fechamento do RFC.

5. Arquitetura do Sistema

Esta seção apresenta como o sistema será construído, utilizando o modelo C4 para descrever a arquitetura em diferentes níveis de abstração.

5.1 Diagrama C4

Nível 1 — Diagrama de Contexto

Visão macro do DocChain como caixa preta dentro do seu ecossistema.

1. **Atores:**
 1. **Usuário** (autenticado) — registra e gerencia seus documentos
 2. **Verificador Público** (visitante) — consulta autenticidade sem autenticação
2. **Sistema Principal:** DocChain (plataforma web)
3. **Sistemas Externos:**
 1. **Sepolia Testnet (Ethereum)** — recebe transações do contrato DocumentRegistry e armazena hashes imutáveis
 2. **Etherscan** — explorador público de transações Ethereum, consultado via deep-link para visualização da tx
4. **Fluxo de Valor:**
 1. Usuário envia arquivo → DocChain processa → registra hash on-chain → retorna confirmação
 2. Verificador envia hash → DocChain consulta blockchain → retorna prova pública

DocChain - Diagrama de Contexto (C4 Nível 1)

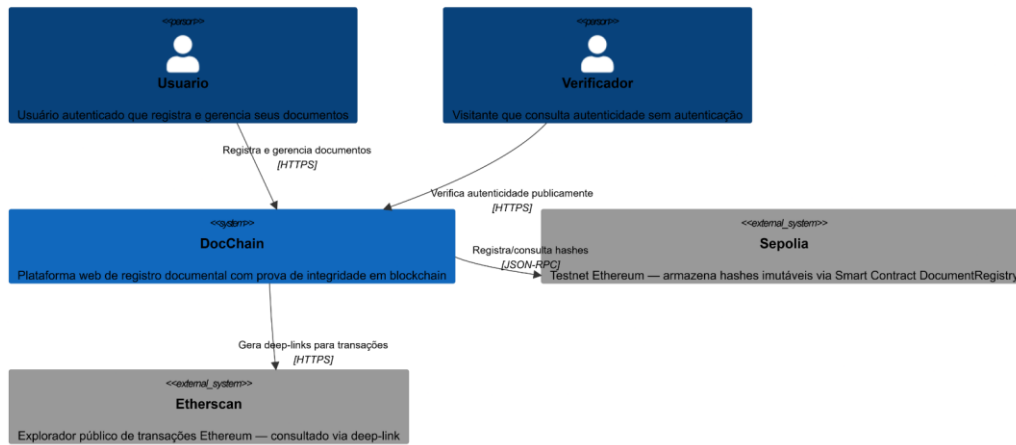


Diagrama C4 Nível 1 — Contexto do sistema DocChain

Nível 2 — Diagrama de Containers

Decomposição do sistema em unidades de execução independentes:

1. **docchain-web** — Aplicação **Next.js 14** com App Router, servida via Node.js. Entrega SSR/CSR para o navegador. Comunica-se via HTTPS/JSON com a API.
2. **docchain-api** — Aplicação **NestJS 11** expondo REST API. Orquestra fluxos de upload, criptografia, storage e blockchain.
3. **docchain-db** — **PostgreSQL 16** armazenando users e documents (apenas metadados).
4. **docchain-storage** — **Volume Docker** montado em /uploads, armazenando arquivos criptografados.
5. **docchain-contracts** (externo) — Smart Contract **DocumentRegistry.sol** implantado na Sepolia. Acessado via Ethers.js + provider RPC (Infura/Alchemy).

Protocolos de comunicação:

Origem	Destino	Protocolo
Browser	docchain-web	HTTPS
docchain-web	docchain-api	HTTPS / JSON (REST)
docchain-api	docchain-db	TCP / PostgreSQL wire protocol (porta 5432)
docchain-api	docchain-storage	Filesystem local
docchain-api	Sepolia (via RPC)	HTTPS / JSON-RPC

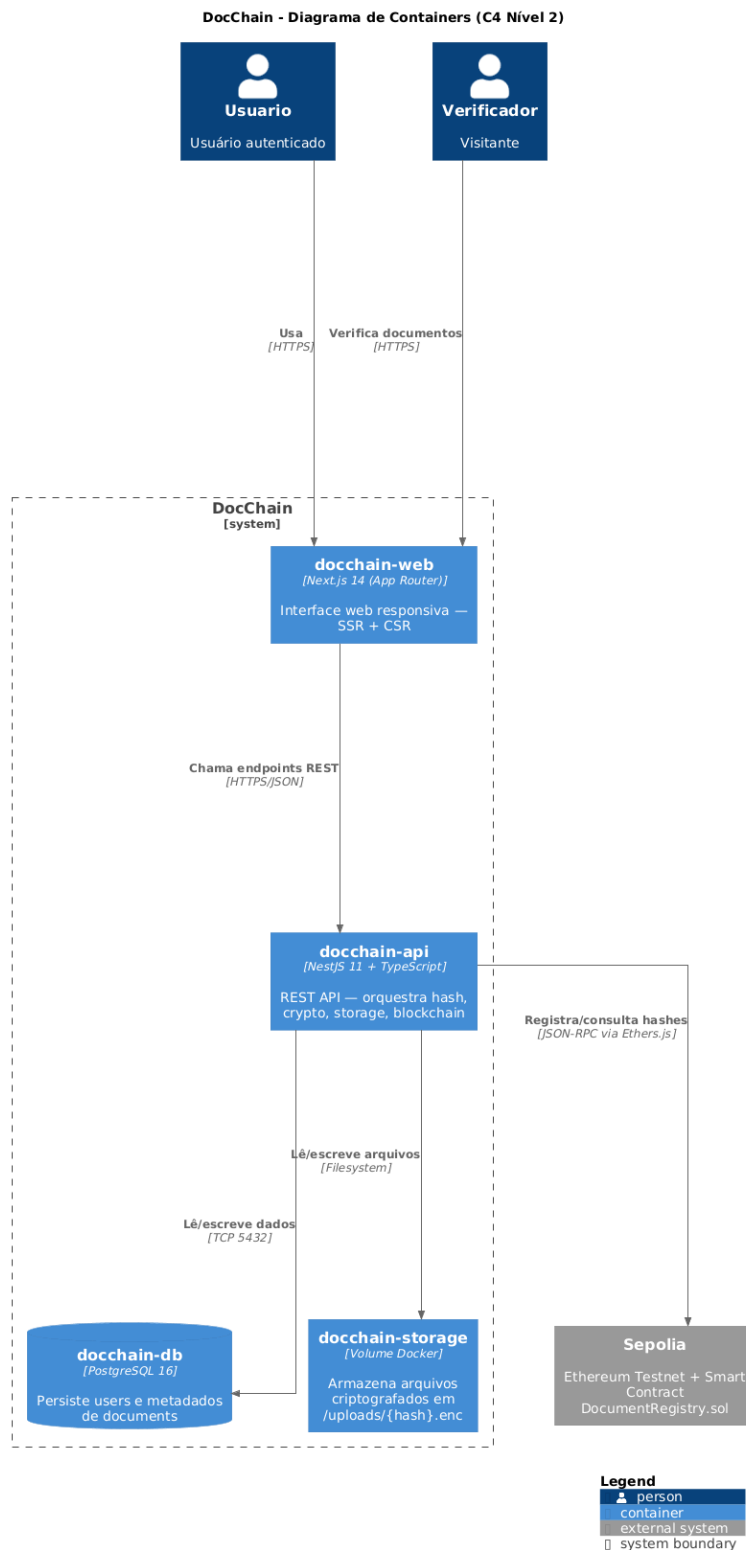


Diagrama C4 Nível 2 — Containers do sistema DocChain

Nível 3 — Diagrama de Componentes (foco no docchain-api)

Estrutura interna do backend NestJS, organizado em módulos:

1. AuthModule

1. AuthController (POST /auth/register, POST /auth/login)

2. AuthService (bcrypt para hash de senha; JwtService para emitir tokens)
3. JwtStrategy + JwtAuthGuard (Passport)

2. DocumentsModule

1. DocumentsController (rotas POST/GET/DELETE de /documents e /verify)
2. DocumentsService (orquestra o fluxo hash → crypto → storage → blockchain)

3. CryptoService (provider interno)

1. hashFile(buffer) → SHA-256
2. encrypt(buffer) / decrypt(ciphertext, iv, authTag) → AES-256-GCM

4. StorageModule

1. IStorageService (interface)
2. LocalStorageService (implementação filesystem em /uploads)

5. BlockchainModule

1. BlockchainService (Ethers.js: registerDocument, verifyDocument, isRegistered)

6. PrismaModule (global)

1. PrismaService (singleton do cliente Prisma)

7. Infraestrutura transversal

1. GlobalExceptionHandler (filtro global de erros)
2. ThrottlerGuard (rate limiting)
3. SwaggerModule (documentação OpenAPI)

5.2 Modelo de Dados

O DocChain utiliza **PostgreSQL** com **quatro entidades** (gerenciadas via Prisma ORM): duas entidades de domínio (User, Document) e duas entidades operacionais para compliance e analytics (AuditLog, VerificationAttempt).

Entidade User:

Atributo	Tipo	Restrições
id	UUID	PK
email	VARCHAR(255)	UNIQUE, NOT NULL
passwordHash	VARCHAR(60)	NOT NULL
name	VARCHAR(120)	NOT NULL
createdAt	TIMESTAMP	NOT NULL, default now()
updatedAt	TIMESTAMP	NOT NULL, auto-updated

Entidade Document:

Atributo	Tipo	Restrições
id	UUID	PK
userId	UUID	FK → User.id, NOT NULL, INDEX
fileName	VARCHAR(255)	NOT NULL
mimeType	VARCHAR(100)	NOT NULL
size	INTEGER	NOT NULL
hash	VARCHAR(64)	UNIQUE, NOT NULL, INDEX
storageRef	VARCHAR(255)	NOT NULL
storageType	ENUM (LOCAL, IPFS)	default LOCAL
encryptionIv	VARCHAR(32)	NOT NULL
encryptionAuthTag	VARCHAR(32)	NOT NULL
status	ENUM (PENDING, PROCESSING, CONFIRMED, FAILED)	default PENDING, INDEX
network	VARCHAR(20)	default 'sepolia'
txHash	VARCHAR(66)	NULLABLE
blockNumber	BIGINT	NULLABLE
walletAddress	VARCHAR(42)	NULLABLE
confirmedAt	TIMESTAMP	NULLABLE
failureReason	TEXT	NULLABLE
createdAt	TIMESTAMP	NOT NULL
updatedAt	TIMESTAMP	NOT NULL

Entidade AuditLog:

Trilha de auditoria das ações sensíveis dos usuários — atende requisitos de **compliance LGPD** (rastreadibilidade) e permite investigação de incidentes de segurança.

Atributo	Tipo	Restrições
id	UUID	PK
userId	UUID	FK → User.id, NULLABLE (NULL para ações públicas), INDEX
action	ENUM (LOGIN, LOGOUT, REGISTER, UPLOAD, DOWNLOAD, DELETE, VERIFY_PUBLIC, VERIFY_PRIVATE)	NOT NULL, INDEX
resourceType	VARCHAR(50)	NULLABLE - "document", "user" etc.
resourceId	UUID	NULLABLE - ID do recurso afetado
ipAddress	VARCHAR(45)	NULLABLE - IPv4 ou IPv6
userAgent	VARCHAR(500)	NULLABLE
metadata	JSONB	NULLABLE - dados específicos da ação (ex: { fileName, hash })
createdAt	TIMESTAMP	NOT NULL, default now(), INDEX

Entidade VerificationAttempt:

Log de toda tentativa de verificação (pública via /verify ou privada via dashboard). Usado para **analytics de uso** do produto, **anti-abuso** (rate limiting baseado em IP) e demonstração de aderência real do mercado.

Atributo	Tipo	Restrições
id	UUID	PK
hash	VARCHAR(64)	NOT NULL - SHA-256 consultado, INDEX
found	BOOLEAN	NOT NULL - true se hash existe on-chain
documentId	UUID	FK → Document.id, NULLABLE, INDEX
userId	UUID	FK → User.id, NULLABLE (NULL para verificação pública)
source	ENUM (PUBLIC, PRIVATE)	NOT NULL, default PUBLIC
ipAddress	VARCHAR(45)	NULLABLE
userAgent	VARCHAR(500)	NULLABLE
createdAt	TIMESTAMP	NOT NULL, default now(), INDEX

Relacionamentos:

- Um **User** possui **muitos Documents** (1:N)
- Um **Document** pertence a exatamente um **User**
- Um **User** gera **muitos AuditLogs** (1:N, cascade SET NULL no delete do user)
- Um **User** gera **muitas VerificationAttempts privadas** (1:N, cascade SET NULL)

5. Um **Document** pode ter **muitas VerificationAttempts** associadas (1:N, cascade SET NULL)



Diagrama Entidade-Relacionamento — modelo de dados do DocChain (4 tabelas)

5.3 Principais Componentes

Backend (docchain-api):

1. **AuthModule** — Cadastro, login e gestão de tokens JWT
2. **DocumentsModule** — Endpoints REST para upload, listagem, detalhe, verificação e download
3. **CryptoService** — Hash SHA-256 e criptografia/descriptografia AES-256-GCM
4. **StorageModule** — Camada de persistência de arquivos com interface abstrata (futura troca por IPFS sem refactor)
5. **BlockchainModule** — Integração com Smart Contract via Ethers.js (registerDocument, verifyDocument, isRegistered)
6. **PrismaModule** — Acesso a banco PostgreSQL com cliente tipado

Smart Contract (docchain-contracts):

1. **DocumentRegistry.sol** — Contrato com mapping bytes32 -> DocumentRecord, funções registerDocument, verifyDocument, isRegistered, evento DocumentRegistered

Frontend (docchain-web):

1. **App Router (Next.js 14)** — Rotas estáticas (/login, /register, /verify) e dinâmicas (/documents/[id])
2. **Middleware de Autenticação** — Protege rotas privadas verificando cookie JWT
3. **Zustand Store** — Estado global de autenticação
4. **API Client (axios)** — Comunicação com backend com interceptors automáticos para Bearer token e tratamento de 401
5. **Componentes UI** — DocumentTable, UploadDropzone, VerifyDropzone, StatusBadge, VerificationResult

Infraestrutura:

1. **Docker Compose** — Orquestra PostgreSQL, API e Web em um único comando
2. **Volumes persistentes** — pgdata (banco) e uploads (arquivos criptografados)

5.4 Stack Tecnológica

Camada	Tecnologia	Justificativa
Smart Contracts	Solidity 0.8.24 + Hardhat 2.28	Linguagem padrão de Ethereum; Hardhat oferece ambiente de teste, deploy e console interativo
Blockchain	Ethereum Sepolia Testnet	Testnet gratuita, ampla documentação, suporte a Ethers.js e Etherscan
Lib Blockchain	Ethers.js v6	API moderna, melhor TypeScript, BigInt nativo, bundle menor que web3.js
Backend	NestJS 11 + TypeScript	Arquitetura modular, DI nativa, integração com Passport/JWT/Swagger/Throttler
ORM	Prisma 7	Schema declarativo como documentação viva, migrations automáticas, cliente totalmente tipado
Banco	PostgreSQL 16	Relacional robusto, suporte a UUID, índices avançados, padrão da indústria
Autenticação	bcrypt + JWT (Passport)	bcrypt para hashing de senha; JWT stateless para escalabilidade horizontal

Camada	Tecnologia	Justificativa
Upload	Multer (memoryStorage)	Padrão do NestJS; processamento em memória sem persistência temporária
Criptografia	Node.js crypto (SHA-256 + AES-256-GCM)	Nativo, sem dependência externa, padrão da indústria
Validação	class-validator + Joi	class-validator para DTOs; Joi para validar env vars
Documentação API	Swagger / OpenAPI	Documentação automática a partir de decorators NestJS
Frontend	Next.js 14 (App Router) + TypeScript	Server Components, SSR/CSR híbrido, melhor DX
Estilização	Tailwind CSS + shadcn/ui	Utility-first; shadcn fornece componentes acessíveis customizáveis
Estado global	Zustand 4	Boilerplate mínimo, hook-based, ideal para auth state
HTTP Client	axios	Interceptors fáceis para injetar Authorization Bearer e tratar 401
Upload UI	react-dropzone	Drag-and-drop com validação de tipo e tamanho
Notificações	sonner	Toasts modernos e acessíveis
Infraestrutura	Docker + Docker Compose	Ambiente reproduzível; "docker compose up" sobe tudo
Versionamento	Git + GitHub	Versionamento distribuído; GitHub para colaboração e CI futuro

Justificativas-chave:

1. **NestJS x Express puro:** NestJS impõe estrutura modular (modules, controllers, services, providers) e já integra IoC, Swagger, Throttler e Passport — economiza tempo e mantém o código organizado em um projeto acadêmico.
2. **Prisma x TypeORM:** Prisma tem melhor migration story, schema único como fonte de verdade e cliente totalmente tipado, reduzindo bugs em tempo de execução.
3. **Sepolia x Mainnet:** Sepolia é gratuita e tem a mesma arquitetura técnica do mainnet — ideal para TCC. Migração futura para mainnet ou L2 (Polygon, Arbitrum) é direta.
4. **AES-256-GCM x AES-CBC:** GCM é autenticado (authTag detecta adulteração em repouso), consolidado como padrão moderno.
5. **Hash do arquivo original x Hash do criptografado:** Hashear o arquivo original permite verificação pública sem necessidade de descriptografar — essencial para o caso de uso /verify e para a propriedade de “prova pública”.

6. Segurança e Privacidade

A segurança é um requisito de primeira classe no DocChain, pois o produto manipula documentos sensíveis e gera provas públicas de autenticidade. Esta seção descreve as defesas adotadas contra as principais classes de ameaças (OWASP Top 10 2021), o modelo de autenticação e autorização, a estratégia de criptografia e o tratamento de dados pessoais sob a Lei Geral de Proteção de Dados (LGPD).

6.1 Proteção contra OWASP Top 10 (2021)

Cada item abaixo lista a ameaça, a forma como ela se aplicaria ao DocChain e a mitigação implementada:

#	Ameaça (OWASP 2021)	Aplicação ao DocChain	Mitigação Adotada
A01	Broken Access Control	Usuário A consultar/baixar documento do usuário B	Middleware JWT em todas as rotas privadas + verificação <code>document.userId === request.user.id</code> no service antes de qualquer operação (download, delete, detalhe)
A02	Cryptographic Failures	Senha em texto, payload no arquivo legível em repouso, segredos no repositório	<code>bcrypt</code> (cost factor 12) para senha; AES-256-GCM com IV aleatório por arquivo + <code>authTag</code> ; segredos via <code>.env</code> validado por Joi; <code>.env</code> no <code>.gitignore</code>
A03	Injection	SQL injection via parâmetros, XSS em campos exibidos no dashboard	Prisma ORM (queries parametrizadas, sem string concat); React/Next.js escapa output por padrão; <code>class-validator</code> valida todos os DTOs
A04	Insecure Design	Reuso de IV de criptografia, registro do hash do arquivo cifrado (não permite verificação pública)	Decisão arquitetural de hashear o arquivo original + gerar IV criptograficamente aleatório por upload (vide seção 5.4)
A05	Security Misconfiguration	CORS aberto, debug em produção, env mal validado	CORS restrito ao domínio do frontend (whitelist via <code>@nestjs/config</code>); Joi valida todas as variáveis de ambiente no bootstrap; <code>NODE_ENV=production</code> desabilita stack traces detalhados no <code>AllExceptionsFilter</code>
A06	Vulnerable & Outdated Components	Dependências com CVEs conhecidos	<code>npm audit</code> rodando no pipeline GitHub Actions; Dependabot habilitado no repositório; versões fixadas com caret (^) em todos os <code>package.json</code>
A07	Identification & Authentication Failures	Força bruta em <code>/auth/login</code> , sessões eternas	Rate limit via <code>@nestjs/throttler</code> aplicado globalmente; expiração curta do access token JWT (<code>JWT_EXPIRES_IN=15m</code>); senha mínima de 8 caracteres validada pelo <code>LoginDto/RegisterDto</code> via <code>class-validator</code>
A08	Software & Data Integrity Failures	Adulteração do arquivo cifrado em disco; transação on-chain falsificada	<code>authTag</code> do AES-GCM detecta qualquer adulteração ao descriptografar (lança erro); transação na Sepolia é assinada pela wallet do servidor e verificável publicamente via <code>txHash</code> no Etherscan
A09	Security Logging & Monitoring Failures	Falta de trilha de auditoria para incidentes	Entidade <code>AuditLog</code> registra ações sensíveis (LOGIN, UPLOAD, DOWNLOAD, DELETE, VERIFY_PUBLIC, VERIFY_PRIVATE); <code>VerificationAttempt</code> registra consultas públicas; logger nativo do NestJS emitindo para <code>stdout</code> (capturado pelo Docker)
A10	Server-Side Request Forgery (SSRF)	Backend buscando recurso externo a partir de input do usuário	DocChain não aceita URLs do usuário em nenhum endpoint — apenas upload binário via Multer (<code>memoryStorage</code>); comunicação com Sepolia usa <code>RPC_URL</code> configurada via <code>env</code> , nunca input do usuário

6.2 Autenticação e Autorização

Autenticação (quem é o usuário):

- **Cadastro (POST `/auth/register`):** senha trafega via HTTPS, é validada por `RegisterDto` (mínimo 8 caracteres via `@MinLength(8)`) e armazenada como hash **bcrypt** (cost factor recomendado de 12 — ≈ 250 ms por verificação no hardware-alvo).

- **Login (POST /auth/login):** credenciais validadas contra o hash; em caso de sucesso, retorna no corpo da resposta JSON um **access token JWT** (algoritmo HS256, segredo em JWT_SECRET, expiração definida por JWT_EXPIRES_IN=15m).
- **Transporte do token:** o frontend armazena o token na memória do navegador (estado Zustand) e o injeta automaticamente em todas as requisições privadas via interceptor axios, no header `Authorization: Bearer {access_token}` — formato definido pelo API_SPEC.
- **Estratégia de validação:** JwtStrategy (Passport) extrai o token do header Authorization, valida assinatura e expiração, e popula `request.user` com o payload (`{ sub: userId, email }`).

Autorização (o que o usuário pode fazer):

- **Guard nas rotas privadas:** decorator `@UseGuards(JwtAuthGuard)` aplicado em todos os controllers privados (Documents); rotas públicas (POST /auth/register, POST /auth/login, GET /verify/public/:hash, GET /health) ficam sem o guard explicitamente.
- **Ownership check:** o DocumentsService recebe `userId` (obtido via decorator `@CurrentUser()`) e aplica `WHERE userId = :userId` em toda query do Prisma — garante que um usuário nunca enxergue documentos de outro, mesmo conhecendo o id UUID do recurso. Tentativa de acessar documento alheio retorna 404 (não 403), evitando vazamento de existência.
- **Princípio do menor privilégio:** a wallet do servidor (que assina transações on-chain) recebe apenas o ETH de Sepolia necessário para a operação esperada; sua chave privada (PRIVATE_KEY) reside em variável de ambiente segregada, nunca no banco, nunca no código, nunca em commits.

Verificação pública (sem autenticação):

- Endpoint GET /verify/public/:hash aceita um hash SHA-256 (64 hex chars) como parâmetro de rota e responde se o documento existe on-chain. **Não retorna metadados privados** (fileName, dono, conteúdo) — apenas `{ hash, registered, registeredAt, registeredBy, network, storageRef }`. O registro é feito em VerificationAttempt apenas para analytics e anti-abuso, sem associar a um usuário (`userId = NULL` quando `source = PUBLIC`).

6.3 Criptografia de Dados Sensíveis

A estratégia de criptografia atua em **três camadas**:

1) Em trânsito: - **HTTPS obrigatório** entre browser ↔ web ↔ api em produção (TLS 1.2+). - O JWT trafega exclusivamente no header `Authorization: Bearer`, protegido pelo TLS — nunca em query string, nunca em log de acesso.

2) Em repouso (arquivo): - Antes de gravar em disco, o backend calcula SHA-256(`originalBuffer`) (módulo crypto nativo do Node.js) e **só então** criptografa o conteúdo com **AES-256-GCM**: - Chave: 32 bytes carregados de ENCRYPTION_KEY (hex de 64 caracteres, gerado uma única vez via `node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"`), validada por Joi na inicialização — falha rápido se ausente ou malformada. - IV (Initialization Vector): 12 bytes gerados via `crypto.randomBytes(12)` por upload — **nunca reutilizado** entre arquivos diferentes (reuso de IV em GCM é falha catastrófica que quebra a confidencialidade). - `authTag`: 16 bytes gerados pelo modo GCM e persistidos na coluna `Document.encryptedAuthTag`. Sua verificação no momento da descriptografia detecta qualquer adulteração do arquivo cifrado em disco (defesa A08) — se o arquivo for modificado, o `decipher.final()` lança erro. - Arquivo cifrado salvo em `${UPLOAD_DIR}/{hash}.enc` (volume Docker persistente, fora do diretório do código).

3) Em repouso (banco de dados): - **Senhas:** bcrypt com cost factor 12. - **Tokens JWT:** não persistidos no banco; são stateless e expiram em 15 minutos — não há tabela de sessão para vazar. - **Campos de PII (email, name):** armazenados em texto na tabela User no MVP — decisão consciente, pois (a) precisam ser indexáveis (email UNIQUE) e pesquisáveis para login, e (b) o vetor de ataque é o acesso ao banco, que é mitigado por isolamento de rede Docker e credenciais segregadas (DATABASE_URL apenas no .env do container API). - **Segredos de aplicação (JWT_SECRET, ENCRYPTION_KEY, PRIVATE_KEY):** vivem **exclusivamente** em variáveis de ambiente; o arquivo .env está no .gitignore desde o commit inicial, e o .env.example (commitado) contém apenas placeholders.

Por que hashear o arquivo original e não o cifrado:

Esta é a decisão de segurança mais importante do projeto. O hash registrado na blockchain é do **arquivo claro**, o que permite a **verificação pública sem necessidade de descriptografar nada** — o verificador faz upload do arquivo que possui, o servidor calcula o SHA-256 e compara com o que está on-chain. Hashear o cifrado tornaria a verificação pública impossível, pois cada nova criptografia geraria um IV diferente e portanto um ciphertext diferente, com hash diferente.

6.4 Privacidade e LGPD

O DocChain coleta o mínimo de dados pessoais necessários para autenticação e auditoria, e oferece controles compatíveis com a Lei nº 13.709/2018 (LGPD).

Dados pessoais coletados:

Categoria	Campo	Origem	Finalidade	Base Legal (LGPD)
Identificação	User.email	cadastro	autenticação, recuperação de conta, notificações operacionais	Execução de contrato (Art. 7º, V)
Identificação	User.name	cadastro	personalização da interface	Execução de contrato (Art. 7º, V)
Segurança	User.passwordHash	cadastro (derivado)	autenticação	Execução de contrato (Art. 7º, V)
Operacional	AuditLog.ipAddress, userAgent	requisição HTTP	trilha de auditoria, investigação de incidentes	Legítimo interesse (Art. 7º, IX) e cumprimento de obrigação legal de segurança (Art. 7º, II)
Operacional	VerificationAttempt.ipAddress, userAgent	requisição HTTP	anti-abuso, rate limit, analytics agregadas	Legítimo interesse (Art. 7º, IX)
Conteúdo	Arquivos enviados pelo usuário	upload	registro documental — pertencem ao titular	Execução de contrato + responsabilidade do titular sobre o conteúdo

Dados que NÃO são coletados: CPF, telefone, endereço físico, dados financeiros, biometria — o produto não os requisita em nenhum fluxo.

Como os dados são armazenados:

- Banco PostgreSQL isolado em rede Docker, acessível apenas pela API.
- Arquivos cifrados (AES-256-GCM) em volume persistente.
- Logs de auditoria contêm apenas IP e User-Agent (não conteúdo do arquivo).
- Backup do banco fora do escopo do MVP acadêmico; em produção real, o backup deve ser criptografado e ter política de retenção compatível com LGPD.

Direitos do titular (Art. 18 LGPD) — como exercer:

A API do MVP, conforme planning/API_SPEC.md, expõe os endpoints de autenticação, documentos e verificação. Os direitos do titular previstos no Art. 18 são atendidos pela combinação dos endpoints existentes mais um conjunto de rotas de gestão de conta a ser implementado dentro do marco **M2 — Backend** (vide seção 7), que estendem o AuthModule:

Direito (Art. 18)	Implementação	Status
Confirmação e acesso aos dados	GET /documents lista os documentos do usuário autenticado; rota GET /auth/me retorna User (id, email, name, createdAt)	GET /documents ☒ no API_SPEC; GET /auth/me previsto em M2
Correção	PATCH /auth/me permite alterar name (e-mail é imutável após cadastro para preservar integridade da chave única)	Previsto em M2
Eliminação / anonimização	DELETE /auth/me: (a) remove os arquivos cifrados de \${UPLOAD_DIR} correspondentes a Document.storageRef, (b) deleta registros de Document em cascata via Prisma, (c) anonimiza AuditLog e VerificationAttempt setando userId = NULL (registro operacional preservado, vínculo pessoal removido), (d) deleta User	Previsto em M2
Portabilidade	GET /auth/me/export retorna JSON com User + lista de Document (metadados, hash, txHash, datas)	Previsto em M2
Informação sobre compartilhamento	DocChain não compartilha dados pessoais com terceiros — não há analytics externos, ad networks ou processadores de pagamento no MVP. O único dado que sai do sistema é o hash SHA-256 enviado à Sepolia, que não identifica o titular	☒ por design
Revogação de consentimento	Equivale à eliminação de conta (DELETE /auth/me)	Previsto em M2

Observação: os endpoints marcados como “previsto em M2” devem ser adicionados ao `planning/API_SPEC.md` antes do início da fase 2. Sua especificação completa (DTOs, respostas, códigos de erro) é parte do escopo do marco M2.

O dado imutável on-chain:

Há um ponto de atenção legal importante: o hash SHA-256 e o `walletAddress` do servidor são gravados em uma blockchain pública (Sepolia) e **não podem ser deletados**. Mitigações:

- O hash é uma **função de mão única** — não permite reconstruir o documento original.
- O `walletAddress` é o da **wallet do servidor DocChain**, não do usuário final — portanto on-chain não há PII do usuário.
- O vínculo “este hash pertence ao usuário X” existe **apenas no banco off-chain**, e é eliminado quando o usuário solicita remoção da conta. Após o `DELETE /auth/me`, o registro on-chain permanece, mas torna-se um hash órfão sem associação pessoal recuperável.

Esta arquitetura é coerente com a recomendação da ANPD de que dados imutáveis em blockchain devem ser limitados a referências não-identificantes (hashes, endereços de wallet do operador), preservando-se a possibilidade de rompimento do vínculo identificador na camada off-chain.

7. Planejamento do Projeto

O desenvolvimento do DocChain está organizado em **5 fases sequenciais**, totalizando aproximadamente **14 dias úteis** de trabalho focado. Cada fase entrega um incremento testável e auditável, com critérios de aceite explícitos. O cronograma abaixo considera a defesa do TCC em **dezembro de 2026** e prevê folga para revisão da banca.

7.1 Marcos do Projeto

Marco	Descrição	Entregáveis	Prazo
M0 — Setup e Infraestrutura	Bootstrap dos três repositórios (docchain-contracts, docchain-api, docchain-web), Docker Compose com PostgreSQL, .env de exemplo e CI básico no GitHub Actions.	3 repos inicializados, docker compose up saudável, lint/typecheck verde	Concluído em 2026-05-22 ☒
M1 — Smart Contract	Implementação do DocumentRegistry.sol, suíte de testes Hardhat, deploy na Sepolia e verificação no Etherscan.	Contrato deployado, endereço documentado, $\geq 90\%$ cobertura nos testes	2026-07-05
M2 — Backend (API)	Módulos AuthModule, DocumentsModule, CryptoService, StorageModule, BlockchainModule e PrismaModule; documentação Swagger; testes unitários e e2e.	API funcional localhost:3000, Swagger em /api/docs, cobertura $\geq 70\%$	2026-08-09
M3 — Frontend (Web)	Telas de login, cadastro, dashboard, upload, detalhe do documento e verificação pública; estado global com Zustand; integração completa com a API.	Web em localhost:3001, fluxo end-to-end funcionando, deploy preview na Vercel	2026-09-06
M4 — Integração e Documentação Final	Docker Compose unificado para todos os serviços, scripts de seed para demonstração, README definitivo, vídeo demo de 3 minutos e revisão da RFC com a banca.	Repositório pronto para clone-and-run, documentação fechada, ensaio de defesa realizado	2026-10-04
Banca de TCC	Defesa pública na Católica SC.	Apresentação + demo ao vivo + arguição	2026-12-XX (data oficial da banca)

7.2 Riscos e Mitigações

Risco	Probabilidade	Impacto	Mitigação
Falha de deploy na Sepolia por esgotamento de ETH faucet	Média	Alto (bloqueia M1)	Solicitar ETH em múltiplos faucets (Alchemy, Infura, PoW) com antecedência; manter saldo ≥ 0.5 ETH testnet
Bug crítico de criptografia identificado tardiamente	Baixa	Muito Alto (perda de dados)	Testes unitários cobrindo ciclo encrypt $\rightarrow$ decrypt $\rightarrow$ hash desde o início; revisão pelo orientador em M2
Atraso na fase de frontend por escopo subestimado	Média	Médio	Telas mais simples (login, cadastro, verify) priorizadas; shadcn/ui reduz custo de componentes
Indisponibilidade do provider RPC (Infura/Alchemy)	Baixa	Médio	Configurar fallback RPC público (Sepolia oficial); retry com backoff exponencial na BlockchainService

Risco	Probabilidade	Impacto	Mitigação
Conflito de cronograma com outras disciplinas no semestre	Alta	Médio	Buffer de ~10 dias entre M4 e a banca; commits frequentes para evitar perda de contexto

8. Referências

Lista exclusivamente as fontes efetivamente utilizadas neste documento — normas citadas, ferramentas presentes no `package.json` dos três sub-repositórios, e o modelo institucional adotado.

8.1 Normas, Padrões e Legislação (citados nas seções 2, 5 e 6)

- **BRASIL.** Lei nº 13.709, de 14 de agosto de 2018. *Lei Geral de Proteção de Dados Pessoais (LGPD)*. Citada na seção 6.4. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm
- **NIST.** *FIPS PUB 180-4 — Secure Hash Standard (SHS)*. Define o algoritmo SHA-256 utilizado para hash do arquivo original (seções 5.4 e 6.3). Disponível em: <https://csrc.nist.gov/pubs/fips/180-4/upd1/final>
- **NIST.** *Special Publication 800-38D — Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. Base do AES-256-GCM utilizado para criptografia em repouso (seções 5.4 e 6.3). Disponível em: <https://csrc.nist.gov/pubs/sp/800/38/d/final>
- **IETF.** *RFC 7519 — JSON Web Token (JWT)*. Formato de token utilizado na autenticação (seções 5.3 e 6.2). Disponível em: <https://data.ietf.org/doc/html/rfc7519>
- **OWASP Foundation.** *OWASP Top 10 — 2021*. Referencial das 10 ameaças tratadas na seção 6.1. Disponível em: <https://owasp.org/Top10/>

8.2 Modelo Arquitetural (citado na seção 5)

- **BROWN, Simon.** *The C4 Model for Visualising Software Architecture*. Base dos diagramas C4 níveis 1, 2 e 3 apresentados na seção 5.1. Disponível em: <https://c4model.com/>

8.3 Documentação Oficial das Tecnologias Utilizadas (todas presentes no `package.json` dos repositórios — vide seção 5.4)

- **NestJS 11.** Framework do backend. <https://docs.nestjs.com/>
- **Next.js 14 (App Router).** Framework do frontend. <https://nextjs.org/docs>
- **Prisma 7.** ORM do backend. <https://www.prisma.io/docs>
- **PostgreSQL 16.** Banco relacional. <https://www.postgresql.org/docs/16/>
- **Solidity 0.8.24.** Linguagem do smart contract. <https://docs.soliditylang.org/en/v0.8.24/>
- **Hardhat.** Ambiente de desenvolvimento Ethereum. <https://hardhat.org/docs>
- **Ethers.js v6.** Biblioteca de integração com Ethereum. <https://docs.ethers.org/v6/>
- **bcrypt (Node.js).** Hash de senha. <https://www.npmjs.com/package/bcrypt>
- **Passport (passport-jwt).** Estratégia de autenticação JWT no NestJS. <https://www.passportjs.org/packages/passport-jwt/>
- **Multer.** Middleware de upload multipart. <https://www.npmjs.com/package/multer>
- **class-validator.** Validação declarativa dos DTOs. <https://github.com/typestack/class-validator>
- **Joi.** Validação das variáveis de ambiente. <https://joi.dev/>
- **@nestjs/throttler.** Rate limiting (mitigação A07). <https://docs.nestjs.com/security/rate-limiting>
- **@nestjs/swagger / OpenAPI 3.** Documentação automática da API. <https://docs.nestjs.com/openapi/introduction>
- **Tailwind CSS.** Framework de utilitários CSS. <https://tailwindcss.com/docs>

- **shadcn/ui.** Componentes acessíveis para React. <https://ui.shadcn.com/docs>
- **Zustand.** Estado global do frontend. <https://docs.pmnd.rs/zustand>
- **react-dropzone.** Drag-and-drop de arquivos. <https://react-dropzone.js.org/>
- **sonner.** Toasts/notificações. <https://sonner.emilkowal.ski/>
- **axios.** Cliente HTTP do frontend. <https://axios-http.com/docs/intro>
- **Docker / Docker Compose.** Orquestração local. <https://docs.docker.com/compose/>

8.4 Infraestrutura Blockchain Utilizada (seções 5.1 e 5.2)

- **Ethereum Sepolia Testnet.** Rede de testes pública onde o DocumentRegistry.sol é implantado. <https://ethereum.org/en/developers/docs/networks/#sepolia>
- **Sepolia Etherscan.** Explorador público de transações usado para verificação independente do txHash. <https://sepolia.etherscan.io/>
- **Provider RPC.** Acesso à Sepolia via Alchemy (<https://www.alchemy.com/>) ou Infura (<https://www.infura.io/>), configurado em RPC_URL.

8.5 Repositório do Projeto

- **Monorepo do TCC:** <https://github.com/cursebearer/Projeto-Portifolio> — contém os três sub-projetos (docchain-contracts/, docchain-api/, docchain-web/), o diretório planning/ com os 9 documentos de planejamento e o diretório RFC docs/ com esta RFC e seus artefatos visuais.

8.6 Modelo Institucional Adotado

- **CATÓLICA SC.** *Modelo de RFC — The Portfolio Playbook.* Estrutura de 10 seções seguida por este documento. Disponível em: <https://github.com/CatolicaSC-Portfolio/The-Portfolio-Playbook/blob/main/documentation/RFC/modelo-de-RFC.md>

9. Apêndices

Apêndice A — Artefatos Visuais

Todos os diagramas referenciados ao longo desta RFC estão disponíveis em alta resolução na pasta RFC docs/artefatos_visuais/ do repositório.

Engenharia de Requisitos / Comportamento: - 1 - Diagrama de Casos de Uso (UML).png - 2 - Diagrama de Sequência – Fluxo Principal.png - 3 - Fluxograma – Fluxo Principal.png - 4. Fluxograma – Verificação Pública.png - 5. Diagrama de Atividade – Erros E01 e E02.png - 6. Fluxo de Navegação (entre telas).png

Mockups de UI (telas-chave): - 7 - login.png - 7 - cadastro.png - 7 - dashboard.png - 7 - upload.png - 7 - documento - id.png - 7 - verify doc.png

Arquitetura (modelo C4 + dados): - 8 - C4 – Nível 1 (Contexto).png - 9. C4 – Nível 2 (Containers).png - 10. C4 – Nível 3 (Componentes do docchain-api).png - 11. DER (Diagrama Entidade-Relacionamento).png

Apêndice B — Estrutura do Repositório

```
Projeto-Portifolio/
├── planning/                                # 9 arquivos .md de planejamento completo
│   ├── README.md
│   ├── ARCHITECTURE.md
│   ├── ROADMAP.md
│   ├── TECH_STACK.md
│   ├── DATABASE_SCHEMA.md
│   ├── SMART_CONTRACT.md
│   ├── API_SPEC.md
│   ├── FRONTEND_SPEC.md
│   └── CLAUDE_CODE_GUIDE.md
├── docchain-contracts/                    # Hardhat + Solidity
├── docchain-api/                          # NestJS + Prisma + PostgreSQL
├── docchain-web/                          # Next.js 14 App Router
├── RFC docs/                              # Esta RFC (.md, .pdf, .docx) + artefatos visuais
└── README.md
```

Apêndice C — Variáveis de Ambiente (exemplo)

```
# docchain-api/.env
DATABASE_URL="postgresql://docchain:docchain@docchain-db:5432/docchain"
JWT_SECRET="<gerado via openssl rand -hex 64>"
JWT_EXPIRES_IN="15m"
ENCRYPTION_KEY="<gerado via openssl rand -hex 32>"
UPLOAD_DIR="/uploads"
MAX_FILE_SIZE_MB=50

# Blockchain
RPC_URL="https://eth-sepolia.g.alchemy.com/v2/<key>"
PRIVATE_KEY="<chave da wallet servidora – NUNCA commitar>"
CONTRACT_ADDRESS="<endereço após deploy>"
NETWORK="sepolia"

# Throttling
THROTTLE_TTL=60
THROTTLE_LIMIT=10
```


Apêndice D — Comandos para Rodar o Projeto Localmente

Clonar o repositório

```
git clone https://github.com/cursebearer/Projeto-Portifolio.git
```

```
cd Projeto-Portifolio
```

Subir infraestrutura completa (DB + API + Web)

```
docker compose up -d
```

Rodar migrations do banco

```
docker compose exec docchain-api npx prisma migrate deploy
```

Acessar:

- Web: <http://localhost:3001>

- API: <http://localhost:3000>

- Swagger: <http://localhost:3000/api/docs>

Apêndice E — Glossário Resumido

Termo	Definição
Hash SHA-256	Função criptográfica que produz uma sequência fixa de 64 caracteres hexadecimais a partir de qualquer entrada. Mudar um byte muda o hash inteiro.
AES-256-GCM	Algoritmo de criptografia simétrica autenticada — cifra o conteúdo e gera um authTag que detecta adulteração.
IV (Initialization Vector)	Valor aleatório que torna cada criptografia única, mesmo para o mesmo arquivo.
Smart Contract	Código executado de forma determinística e imutável na blockchain.
Sepolia	Testnet pública do Ethereum, gratuita, com a mesma arquitetura técnica da mainnet.
JWT	JSON Web Token — credencial stateless assinada que representa a sessão do usuário.
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei 13.709/2018).
PII	Personally Identifiable Information — dados que identificam uma pessoa natural.

10. Parecer do Comitê de Avaliação

Seção reservada para preenchimento pelos professores avaliadores.

Avaliador	Status	Observações
Avaliador 1	☐ Aprovado ☐ Ajustar	
Avaliador 2	☐ Aprovado ☐ Ajustar	

Data da avaliação avaliador 1: ____ / ____ / ____

Data da avaliação avaliador 2: ____ / ____ / ____

Assinaturas:

Avaliador 1

Avaliador 2
